



TPanel

Reference guide

Written by Andreas Theofilu <andreas@theosys.at>

© 2022 to 2025 by Andreas Theofilu

Table of contents

Introduction.....	9
Why does TPanel exist?.....	9
Why AMX?.....	10
Old <i>discontinued</i> equipment from AMX.....	10
Still supported devices.....	12
Programming.....	14
Overview.....	14
Touch Gesture Recognition.....	14
Setup Dialog.....	14
Logging.....	15
Android.....	15
iPhone, iPad.....	15
All mobile devices.....	15
Log steps.....	15
Log formats.....	16
Profiling.....	16
Long format.....	16
Logfile.....	16
Controller (NetLinx).....	17
Downloading the surface from the NetLinx.....	17
Controller.....	18
Network port.....	18
Channel number.....	18
Panel type.....	18
FTP user name.....	18
FTP password.....	18
TP4/TP5 file name.....	18
FTP passive mode.....	19
SIP.....	20
Proxy.....	20
Port.....	20
TLSport.....	20
STUN.....	20
Domain.....	20
User.....	20
Password.....	20
Network.....	21
State.....	21
View.....	21
Scaling.....	21
Startup banner.....	22
Toolbar.....	22
Rotation.....	23
Sound.....	24
System sound.....	24
Single beep.....	24

Double beep.....	24
System sound (play system sound).....	24
Volume.....	25
Gain.....	25
Play test sound.....	25
Using TPanel on a mobile device.....	26
Enable APK installs on non Samsung devices.....	26
Enable APK installs on Samsung devices.....	26
TPanel on iOS.....	26
Starting TPanel.....	26
In case of problems.....	27
Things who are not working.....	27
Passwords in resources.....	27
Panel to panel communication.....	28
TakeNotes.....	28
Remote computer control.....	28
Page commands.....	29
@AFP / ^AFP.....	29
@APG.....	29
@CPG.....	29
@DPG.....	29
@PHE.....	30
@PHP.....	31
@PHT.....	31
^POP.....	31
@PPA / ^PPA.....	31
@PPF / ^PPF.....	32
@PPG / ^PPG.....	32
@PPK / ^PPK.....	32
@PPM / ^PPM.....	33
@PPN / ^PPN.....	33
@PPT / ^PPT.....	33
@PPX / ^PPX.....	33
^PTC.....	34
^PTO.....	34
@PSE.....	35
@PSP.....	35
@PST.....	35
PAGE / ^PGE.....	35
PPOF.....	36
PPOG.....	36
PPON.....	36
Programming Numbers.....	37
RGB Triplets and Names For Basic 88 Colors.....	37
Font Styles And ID Numbers.....	38
Border Styles And Programming Numbers.....	39
Button Query Commands.....	40
^ANI.....	41
^APF.....	41
^BAT.....	41

^BAU.....	42
^BCB.....	42
?BCB.....	43
^BCF.....	43
?BCF.....	44
^BCT.....	44
?BCT.....	45
^BDO.....	45
^BFB.....	46
^BIM.....	46
^BMC.....	47
^BMF.....	47
^BML.....	49
^BMP.....	50
?BMP.....	51
^BOP.....	51
?BOP.....	52
^BOR.....	52
^BOS.....	53
^BRD.....	53
?BRD.....	54
^BSM.....	54
^BSO.....	54
^BSP.....	55
^BWW.....	55
?BWW.....	55
^CPF.....	56
^DPF.....	56
^ENA.....	56
^FON.....	57
?FON.....	57
^GDI.....	57
^GLH.....	58
^GLL.....	58
^GRD.....	58
^GRU.....	58
^GSC.....	59
^GSN.....	59
^ICO.....	59
?ICO.....	60
^JSB.....	60
?JSB.....	61
^JSI.....	61
?JSI.....	62
^JST.....	62
?JST.....	63
^MSP.....	63
^SCE.....	63
^SHA.....	65
^SHD.....	66

^SHO.....	66
^SPD.....	66
^SSH.....	67
^STG.....	68
^TEC.....	68
?TEC.....	69
^TEF.....	69
?TEF.....	70
^TXT.....	70
?TXT.....	71
^UNI.....	71
^UTF.....	72
^VTP.....	72
MVP Panel Lock Passcode Commands.....	73
^LPB.....	73
^LPC.....	73
^LPR.....	73
^LPS.....	74
Text Effect Names.....	75
Panel Runtime Operations.....	76
@AKB.....	76
AKEYB.....	76
AKEYP.....	76
AKEYR.....	76
@AKP.....	77
@AKR.....	77
ABEEP.....	77
ADBEEP.....	77
BEEP / ^ABP.....	77
DBEEP / ^ADB.....	77
@EKP / ^EKP.....	78
^ENC.....	78
?ENC.....	79
?MAC.....	80
^MUT.....	80
PKEYP.....	81
@PKP.....	81
^RPP.....	81
SETUP / ^STP.....	81
SHUTDOWN.....	81
@SOU / ^SOU.....	82
@TKP / ^TKP.....	82
@VKB.....	82
Input Commands.....	83
^KPS.....	83
^VKS.....	83
Dynamic Image Commands.....	84
^BBR.....	84
^RAF.....	84
^RFR.....	84

^RMF.....	85
^RSR.....	85
Subpages commands.....	86
^EPR.....	86
^LVD.....	86
^LVE.....	87
^LVF.....	88
^LVL.....	88
^LVM.....	90
^LVN.....	91
^LVR.....	92
^LVS.....	93
^SCE.....	94
^SDR.....	95
SIP Commands.....	96
Panel to Master.....	96
^PHN-AUTOANSWER.....	96
^PHN-CALL.....	96
^PHN-IM.....	96
^PHN-INCOMING.....	96
^PHN-LINESTATE.....	97
^PHN-MSGWAITING.....	97
^PHN-PRIVACY.....	97
^PHN-REDIAL.....	97
^PHN-TRANSFERRED.....	98
Master to Panel.....	98
^PHN-ANSWER.....	98
^PHN-AUTOANSWER.....	98
?PHN-AUTOANSWER.....	98
^PHN-CALL.....	99
^PHN-DECLINE.....	99
^PHN-DTMF.....	99
^PHN-HANGUP.....	99
^PHN-HOLD.....	99
^PHN-IM.....	100
?PHN-LINESTATE.....	100
^PHN-PRIVACY.....	100
?PHN-PRIVACY.....	100
^PHN-REDIAL.....	100
^PHN-TRANSFER.....	101
^PHN-SETUP-DOMAIN.....	101
^PHN-SETUP-ENABLE.....	101
^PHN-SETUP-PASSWORD.....	101
^PHN-SETUP-PORT.....	101
^PHN-SETUP-PROXYADDR.....	102
^PHN-SETUP-USERNAME.....	102
Commands similar to <i>TPControl</i>	102
TPCCMD-LocalHost.....	102
TPCCMD-LocalPort.....	102
TPCCMD-DeviceID.....	103

TPCCMD-ApplyProfile.....	103
TPCCMD-QueryDeviceInfo.....	103
TPCCMD-LockRotation.....	103
TPCCMD-ButtonHit.....	103
TPCCMD-ReprocessTP4.....	104
TPCACC.....	104
Appendix A.....	105
Text area input masking.....	105
Input mask character types.....	105

Abbildungsverzeichnis

Picture 1: Log settings (desktop).....	14
Picture 2: Controller (NetLinx) settings (desktop).....	16
Picture 3: Question to download a file.....	18
Picture 4: Install surface?.....	18
Picture 5: SIP settings.....	19
Picture 6: Optical settings.....	20
Picture 7: Navigation wheel on an MVP-5200i.....	21
Picture 8: Toolbar of TPanel.....	21
Picture 9: About dialog on a mobile device.....	22
Picture 10: Sound settings (desktop).....	23

Introduction

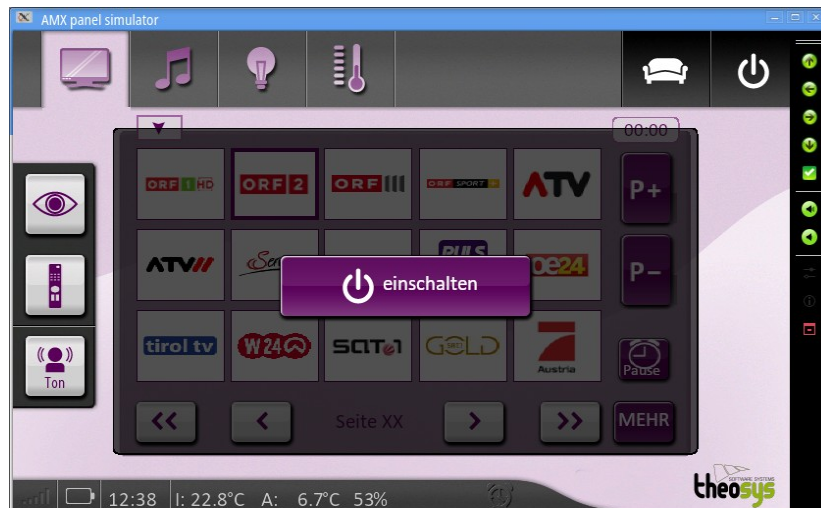
TPanel is an implementation of some AMX G4/G5 touch panels. The panels used to verify the proper communication protocol and the behavior were an *AMX MVP-5200i*, *AMX NXD-700Vi* and an *AMX MST-701* and some other G5 panels.

This manual describes the commands implemented and some specials of this program. **TPanel** was designed for *NIX desktops (Linux, MacOS, BSD, ...) as well as Android and iOS operating systems. Currently there exists no Windows version and there probably never will.

The software uses internally the [Skia](#)¹ library for drawing all objects and the [Qt 6.x](#) library to display the objects. **TPanel** is written in C++. This makes it even on mobile platforms fast and reliable. It has the advantage to not drain the battery of any mobile device while running as fast as possible. Compared to commercial products the battery lasts up to 10 times longer.

Why does TPanel exist?

I'm a professional software engineer and years ago I got in touch with AMX. I developed solutions for residential requirements in NetLinx with different AMX panels. With time the customers wanted to have the surface on their phone or a tablet and there was (is?) only one solution available on the market. While this software is very good and supports everything up to TP5 commands, the license is rather expensive. For me as a private person, too expensive. But I bought some used AMX NetLinx on the Internet and build my own smart home where I can control lights, hazard and my consumer electronics (TV, radio, audio, video, ...). I bought also some used AMX panels but had no luck with the batteries in them. To buy a new battery from AMX was too expensive and it didn't pay off for such old devices. Buying a license for the commercial version (TPControl) was also no option because I would need 4 of them. So I decided to program my own surface and it should behave as a real device from AMX.



Picture 1: Main window with my surface

¹ Skia is an open source 2D graphics library which provides common APIs that work across a variety of hardware and software platforms. It serves as the graphics engine for Google Chrome and Chrome OS, Android, Flutter, and many other products. Skia is sponsored and managed by Google, but is available for use by anyone under the BSD Free Software License.

Why AMX?

There are a lot of possibilities to build a smart home. Makers can use a Raspberry PI or an Arduino or something similar. You can create circuits to make serial ports, infrared control, I/O ports and relays. This is a lot of effort and you may spent up to several hundred Euros to build just a controller with features like AMX NetLinx already have out of the box.

On the other side you can control your lights, the hazard and some modern TVs over Alexa and co. While this works it has the disadvantage to need an internet connection and there is a big cloud behind. If the internet is not available for whatever reason, you can't control anything. Beside that it exposes your devices to the Internet and you never know who may access them. This is a security whole and you should take care of it.

For me some essential points are that I want to be absolutely independent of any internet connection and any cloud. I wanted to have a system which needs only a local network. Therefor I need a controller handling the smart home. It should be cheap and easy to handle. Since [AMX](#) offers all the necessary software to program without the need of a company account it is easy to get the knowledge to program an AMX [NetLinx](#). Beside this it is really easy to get a used AMX NetLinx from eBay. I bought mine for about 50 Euros. Even some older equipment like volume controllers (AXB-VOL-3) are still available on eBay. With a *NI-3100* NetLinx you can do everything you need to make your home smart. If you like, you can try to find some G4 or G5 panels also on the internet and you *will* find them.

To make it short: AMX is a fast and cheap way to implement just the software to make a smart home if you buy used equipment. In most cases you need no additional hardware (beside a local network). On the other side you must be interested in programming and you must be willing to learn an easy 3rd generation language like [NetLinx](#) is. This are the reasons for me to use AMX.

Old *discontinued* equipment from AMX

The following table shows some older devices from AMX who are not continued any more but still available on eBay and therefor are cheap. This devices are still fully functional and will be enough for most requirements.

Device	Description
NI-700	The AMX NI-700 was designed to meet the needs of single room requirements while keeping cost in mind all in a 1RU box. The unit can control a limited number of video players, projectors, lights, thermostats, and other electronic equipment. The NI-700 is ideal for classrooms, conference rooms, hotel rooms and so much more. Includes: 2-pin 3.5 mm mini-Phoenix female PWR connector, 4-pin 3.5 mm mini-Phoenix female connector, 6-pin 3.5 mm mini-Phoenix female I/O connector, CC-NIRC IR Emitter

Device	Description
NI-900	The AMX NI-900 was created to automate and control numerous items in 1 large room or several small rooms. The NI-900 is ideal since it can support several different devices with numerous different communication formats. Common applications include hotel rooms, home theaters, and other environments. The NI-900 is configured to control a small number of lights, thermostats, flat panels, and other audio video equipment. Includes: 2-pin 3.5 mm mini-Phoenix female PWR connector, 6-pin 3.5 mm mini-Phoenix female I/O connector, Three CC-NIRC IR Emitters, Two 4-pin 3.5 mm mini-Phoenix female connectors.
NI-2100	The AMX NI-2100 was designed for the automation and control of medium sized rooms and multiple room applications. The unit features 64MB of RAM and 3 configurable RS-232 / RS-422 / RS-485 serial ports. Programming the NI-2100 is simple since it is device discovery enabled offering several functions definitions for standardizing devices as well as default touch panel button assignments, and control and feedback methods. Includes: 2-pin 3.5 mm mini-Phoenix (female) PWR connector, 4-pin 3.5 mm mini-Phoenix (female) AxLink connector, 6-pin 3.5 mm mini-Phoenix female I/O connector, 8-pin 3.5 mm mini-Phoenix female Relay connector, Two CC-NIRC IR Emitters, Two removable rack ears.
NI-3100	The AMX-3100 was designed for large rooms or even multiple rooms where you need the ultimate control and automation. The controller can control numerous items including audio/video conferencing, projectors, DVD and Blu-Ray players, lights, thermostats and other electronic equipment found in larger rooms. Not only can it accomplish what you need now it can also provide solutions for future needs with its easy expansion capabilities. Installation is easy with device discovery enabled and performance is top notch with the speedy processor and 64MB of RAM. Includes: 2-pin 3.5 mm mini-Phoenix (female) PWR connector, 4-pin 3.5 mm mini-Phoenix (female) AxLink connector, 10-pin 3.5 mm mini-Phoenix (female) I/O connector, Two 8-pin 3.5 mm mini-Phoenix female Relay connectors, Two CC-NIRC IR Emitters, Two removable rack ears.
NI-4100	The NI-4100, part of the NI Series of Master Controllers, is geared to meet the high-end control and automation requirements of the most sophisticated and complex commercial and residential installations. This controller integrates the largest number of devices including DVD players, projectors, lighting, thermostats and other electronic equipment. In technology-intensive environments, this solution can be used to accommodate the future addition of more devices and control capabilities
AXB-VOL-3	The AXB-VOL3 Three-Channel Volume Control provides three audio volume control channels. Each line-level channel, opto-isolated from system ground, can be configured for balanced or unbalanced line operation. The AXB-VOL3 is programmable for 128 steps of audio level, audio mute, variable ramp speed and level presets. The AXB-VOL3 connects to NetLinx control systems using the 4-wire AXlink data/power bus; it can be used for remote or rack mount applications.

Device	Description
NXC-VOL4	NXC-VOL4 by AMX offers four discrete volume control channels with LED feedback and is programmable for mono or stereo operation, and balanced or unbalanced audio connections. Programmed features such as audio levels, audio mute, variable ramp speeds and preset levels. Use the on-board jumpers to set the gain/attenuation (Unity, Pro level (+4 dBu) to Consumer level (-10 dBu) conversion, or Consumer level to Pro level on each channel). NetLinx Control Cards provide flexible, modular building blocks for creating advanced control applications.
EXB-COM2	ICSLan Device Control Boxes allow users to manage devices remotely from a Controller over an Ethernet network. This provides a beautifully simple method for a centralized control environment allowing users to share a controller among multiple smaller rooms versus controllers in every room. Ethernet has become the industry standard for connecting devices and the ICSLan Device Control Boxes make it easy to introduce control to equipment such as projectors located extended distances from a Controller. Additionally, the number of ports on an AMX Controller can be expanded when all ports are fully populated. Because they employ Native NetLinx technology, it is extremely simple to add an EXB to an AMX installation.

Still supported devices

The following table shows some useful devices supported by AMX but also available on eBay. They may be more expensive but if you want the latest stuff this is for you.

Device	Description
NX-1200	The NX-1200 / NetLinx® NX Integrated Controller is a programmable network appliance specifically designed to control AV and building technology using multiple analog and digital formats. The NX-1200 provides a scalable platform for the future by combining high performance, backward compatibility and extensive network security features. The NX-1200 is designed to control and automate a variety of devices in single rooms or multiple small rooms and is capable of supporting devices with a variety of communication formats.
NX-2200	The NX-2200 / NetLinx® NX Integrated Controller is a programmable network appliance specifically designed to control AV and building technology using multiple analog and digital formats. The NX-2200 provides a scalable platform for the future by combining high performance, backward compatibility and extensive network security features. The NX-2200 is ideal for control and automation of medium-sized rooms and multi-room applications, as it includes more ports and security capabilities than the NX-1200.

Device	Description
NX-3200	The NX-3200 / NetLinx® NX Integrated Controller is a programmable network appliance specifically designed to control AV and building technology using multiple analog and digital formats. The NX-3200 provides a scalable platform for the future by combining high performance, backward compatibility and extensive network security features. The NX-3200 adds more ports to the NX-2200, giving it the ability to integrate a larger number of devices. The NX-3200 controls a wide variety of components including audio/video conferencing, projectors, DVD and Blu-ray players, lights, thermostats and other electronic equipment found in larger spaces. With these technology-driven environments, the NX-3200 also provides solutions for future expansion and enables the addition of more devices and control capabilities.
NX-4200	The NX-4200 / NetLinx® NX Integrated Controller is a programmable network appliance specifically designed to control AV and building technology using multiple analog and digital formats. The NX-4200 provides a scalable platform for the future by combining high performance, backward compatibility and extensive network security features. With its front panel display, built-in Ethernet switch and PoE capability, the NX-4200 is the ideal solution for demanding, technology-rich environments. The NX-4200's all-in-one architecture also delivers the lowest Total Cost of Ownership in the industry, thanks to ease of support, maintenance and configuration, as well as reduced hardware and cabling costs.

The devices above were under support at the time this manual was written!

Programming

Overview

You can program **TPanel** using the commands in this section, to perform a wide variety of operations using `Send_Commands` and variable text commands.

A device must first be defined in the *NetLinx* programming language with values for the Device: Port: System (in all programming examples - Panel is used in place of these values and represents **TPanel** program).

Touch Gesture Recognition

TPanel supports currently the pinch gesture and the swipe gestures. With a pinch gesture it is possible to open the setup dialog. It can be used on any device with a touch screen. The swipe gestures are send as such to a NetLinx and it's up to you to do something with it.

Setup Dialog

The setup dialog allows the setting of different things. It's look and feel depends on the operating system. On a desktop it is a dialog box and consists of *tabs* on the top allowing to select the wanted page.

On iOS and Android the native setup dialogs are used. While on iOS the setup is separated into subpages, this is not supported on Android yet (although Android offers this possibility).

Currently 5 pages are available (sections on Android):

- Logging → Settings for the logfile.
- Controller → Everything about the AMX NetLinx.
- SIP → *Session Initiation Protocol* used for phone calls.
- View → Some settings about visual effects.
- Sound → Some sound settings.

The following sections show the setup dialog as it is on a desktop. The look differs on mobile devices but the possible selections are the same.

Logging

The logging is meant to be enabled in case of problems. For example if you find a situation where the program crashes. In such a case logging can help the developers to find the course for the crash.

Android

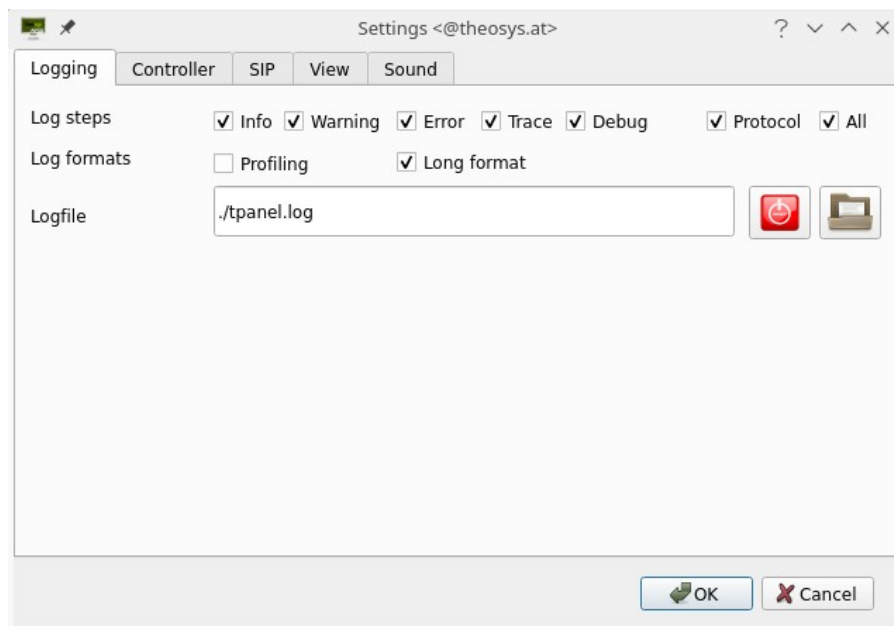
On mobile devices logging is disabled by default. The reasons are, that a special permission to the disc is necessary and by default the logfile is in a place where the user have no access to it. Enabling logging means also to put the logfile somewhere on the disc where a user has access to it. This can easily be done with a file dialog. Defining the path and name of the logfile also asks for permissions in case they are not already granted.

iPhone, iPad

Although you can enable logging, the file will not be accessible due to restrictions on the file system. So logging in this devices is useless!

All mobile devices

Since version 2.0.0 only the logging options *Info*, *Warning*, *Error* and *Protocol* are possible. Although you can choose *Debug* and *Trace* too, they will have no effect! This is because a *Release* build disables debugging by not building the corresponding code. On a desktop system the default build is always *Debug*.



Picture 1: Log settings (desktop)

Log steps

The logging is based on states. This means that every stage of logging can be enabled or disabled independently of the other stages. There exists also two shortcuts: *Protocol* and *All*.

The *Protocol* stage is a combination of *Info*, *Warning* and *Error*.

The *All* stage enables all stages. This produces a lot of output and should not be used on a mobile device. When this is enabled everything is logged into a file. Because the program uses a lot of threads inside the content of the logfile may look funny sometimes. To be able to follow a thread, the thread ID is part of the logfile.

If you're not a developer I suggest to disable logging at all. Especially on a mobile device.

Log formats

Profiling

If this is enabled together with the *Trace* log option, a time stamp is printed at the end of each method.

```
TRC    75,    {entry TExpat::parse()
TRC    34,    {entry TValidateFile::isValidFile(const string& file)
TRC      ,    }exit TValidateFile::isValidFile(const string& file) Elapsed
time: 2508[ns] --> 0s 0ms
TRC      ,    Parsing XML file /usr/share/tpanel/map.xma
TRC      ,    }exit TExpat::parse() Elapsed time: 43929457[ns] --> 0s 43ms
TRC   318,    {entry TExpat::getElementIndex(const string& name, int* depth)
TRC      ,    }exit TExpat::getElementIndex(const string& name, int* depth)
Elapsed time: 4511[ns] --> 0s 0ms
```

The elapsed time is printed in nanoseconds as well as in seconds and milliseconds.

Long format

This adds additional information like a time stamp. If *Trace* is enabled you'll see also the file name where the class is located along with the line number where the message was executed from.

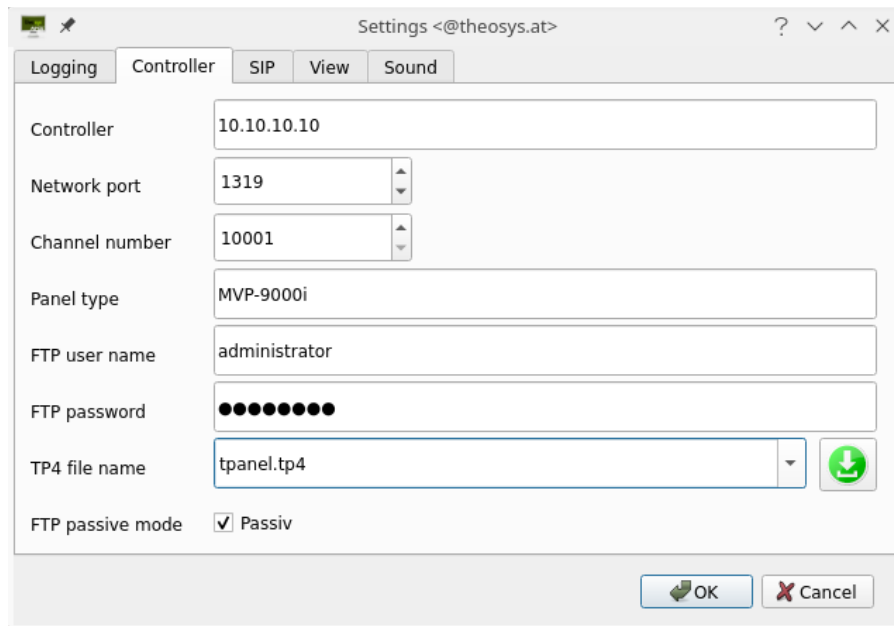
```
2023-04-03 18:13:19 TRC    95, tsipclient.cpp    , 7ff6cf9169c0 {entry TSIPClient::TSIPClient()
2023-04-03 18:13:19 TRC   970, tconfig.cpp    , 7ff6cf9169c0 {entry TConfig::getSIPstatus()
2023-04-03 18:13:19 TRC      , tconfig.cpp    , 7ff6cf9169c0 }exit TConfig::getSIPstatus() Elapsed time: 417[ns] --> 0s 0ms
2023-04-03 18:13:19 TRC      , tsipclient.cpp , 7ff6cf9169c0 }exit TSIPClient::TSIPClient() Elapsed time: 42598[ns] --> 0s
0ms
2023-04-03 18:13:19 TRC  3969, tpagemanager.cpp , 7ff6cf9169c0 {entry TPageManager::runClickQueue()
2023-04-03 18:13:19 TRC      , tpagemanager.cpp , 7ff6cf9169c0 }exit TPageManager::runClickQueue() Elapsed time: 76981[ns] -->
0s 0ms
2023-04-03 18:13:19 TRC  4017, tpagemanager.cpp , 7ff6cf9169c0 {entry TPageManager::runUpdateSubViewItem()
2023-04-03 18:13:19 PRT    ++,                , 7ff6cf9136c0 Thread "TPageManager::runClickQueue()" was started.
2023-04-03 18:13:19 TRC      , tpagemanager.cpp , 7ff6cf9169c0 }exit TPageManager::runUpdateSubViewItem() Elapsed time:
50975[ns] --> 0s 0ms
2023-04-03 18:13:19 TRC      , tpagemanager.cpp , 7ff6cf9169c0 }exit TPageManager::TPageManager() Elapsed time: 4109395594[ns]
--> 4s 109ms
2023-04-03 18:13:19 TRC    94, tlock.cc        , 7ff6cf9169c0 {entry TLock<TMutex>::~TLock()
2023-04-03 18:13:19 TRC   308, tlock.cc        , 7ff6cf9169c0 {entry TLock<TMutex>::~removeLock(__LOCKLIST_t& e)
2023-04-03 18:13:19 PRT    ++,                , 7ff6cf1126c0 Thread "TPageManager::runUpdateSubViewItem()" was started.
2023-04-03 18:13:19 DBG    --,                , 7ff6cf9169c0 Lock for mutex handle 0x558c84eb2138 will be removed on file
tpagemanager.cpp at line 701.
2023-04-03 18:13:19 TRC      , tlock.cc        , 7ff6cf9169c0 }exit TLock<TMutex>::~removeLock(__LOCKLIST_t& e) Elapsed time:
182171[ns] --> 0s 0ms
2023-04-03 18:13:19 TRC      , tlock.cc        , 7ff6cf9169c0 }exit TLock<TMutex>::~TLock() Elapsed time: 215103[ns] --> 0s
0ms
```

Logfile

This line defines where the log should be written. Select a path and name for the logfile. If you want to see some logs on a mobile device (phone, ...) you must make sure that the file is written somewhere in the user space. By default the log file is set to the internal directory where the program itself is stored. This directory is accessible only by the program!

Controller (NetLinx)

The controller page let you set everything about the NetLinx. There is the IP address the controller is listening on and you may define the FTP credentials to download TP4 files directly from the NetLinx.



Picture 2: Controller (NetLinx) settings (desktop)

Downloading the surface from the NetLinx

From version 1.3.0 on it is possible to put one or more normal TP4 files on the disc of the NetLinx. The name of the files doesn't matter, because the settings dialog is reading the directory on the NetLinx and show all files found with an extension of *TP4* and *TP5*. If no TP4 or TP5 – file was found on the NetLinx, the default file `tpanel.tp4` is set in the settings dialog. Of course this works only if a valid IP address or a network name was defined. Additionally the FTP access of the NetLinx must be enabled.

If **TPanel** is started and there is no surface found, it tries to connect to the NetLinx via FTP (File Transfer Protocol) with the credentials defined in the settings. If it succeeds and a file is found, then it downloads the file and unpacks it. Afterward **TPanel** loads the fresh surface and displays it.

If there is already a surface available and the name of the surface file in the settings dialog is changed, it tries to download this new file. If it succeeds it deletes the old surface and unpacks the new one. Afterward it restarts itself.

On a desktop as well as on Android this is working. On iOS the setup dialog doesn't support run time settings from outside. Therefor you must know the name of the TP4-file and write it into the input line. When the setup returns to the application, the surface will be downloaded as on any other platform.

Controller

Enter in this field either the network name of the NetLinx or the IP address. If this field contains no valid address the program is not able to connect the NetLinx. This is also necessary to enable **TPanel** to access the disc of the NetLinx to search for TP4 and TP5 files.

***Note:** Currently only plain access is possible. **TPanel** doesn't allow encrypted access to a NetLinx!*

Network port

Enter the network port number the NetLinx is listening on. If not changed in the setup of the NetLinx this is port 1319.

Channel number

Enter the channel number of the panel. This must be a number between 10000 and 19999. It must be a unique number. This means that no other panel must have this number.

Panel type

Enter here the description of the panel. This should be a name supported by *TPDesign4/5*. For example a valid name would be MVP-5200 or VARIA-SL80.

FTP user name

This defines the user name of the FTP user. By default this is set to **administrator**. The user name is used to automatically login to the NetLinx and look for a TP4/TP5 file.

FTP password

This defines the password needed to logon to the NetLinx via FTP. By default this is set to **password**. Set this to the password the NetLinx expects.

TP4/TP5 file name

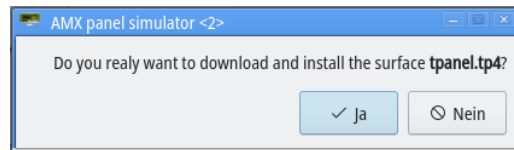
This defines the name of a TP4/TP5 file. Such a file can be downloaded automatically from **TPanel**. If the FTP credentials (user name and password) are correct and there is at least one TP4 or TP5 file on the disc of the NetLinx, then **TPanel** downloads the file, unpacks it and installs it. After an automatic restart the new surface is visible (may take up to several minutes depending on the speed of the device!).

This so called combo box is the combination of an edit line and a list. When the settings dialog is opened, **TPanel** tries to connect to the NetLinx with the given FTP credentials. If it succeeds it reads the root directory of the NetLinx. It puts each file with an extension of TP4 or TP5 into this element. When you click on the small arrow on the right end of the line, it opens up and shows the content, if any. While this is true for desktops, it works similar for Android. The look and feel depends on the flavor of your phone.

On iOS this doesn't work. There it is a simple input line where you can type the name of the file. File names are case sensitive. This means that you must take care about capital and small letters!

If there were no TP4 files found on the NetLinx then it contains only the default file name `tpanel.tp4` or `tpanel.tp5`.

On the right end of this line is a button with a down arrow visible. If you click on it a dialog box opens (not true for Android and iOS!).



Picture 3: Question to download a file.

If you want to download this file click on YES. Otherwise NO. If you select YES, then the force button gets a red background. This means that the download starts at the moment the settings dialog is closed.

If you select another surface file and close the dialog, **TPanel** ask you if you want to download the new surface.



Picture 4: Install surface?

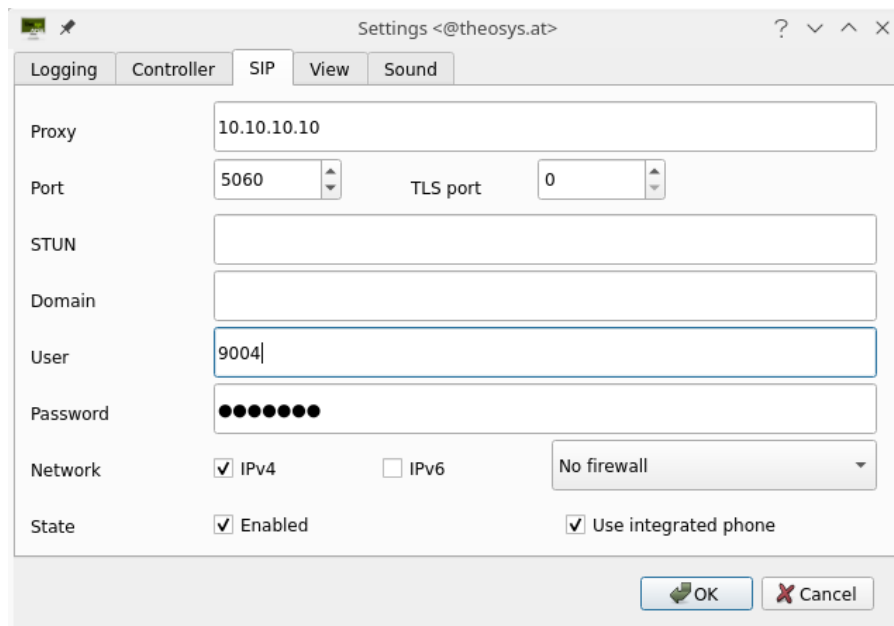
If you click on YES the download starts. Otherwise nothing happens.

FTP passive mode

The FTP protocol knows two different methods to establish a data channel to transfer data from a computer. The default mode is called *port* and requires that your client has full access to the computer. This works as long as there is no firewall in between who blocks the network port 20. To overcome any firewalls, the protocol knows a mode called *passive*. In this case the client connects to a second channel the same way as it did with the control channel. This makes sure that the client can connect even if there is a firewall between. In doubt check this option.

SIP

This is a protocol to connect **TPanel** to a digital phone. There is a full soft phone implemented into **TPanel**.



Picture 5: SIP settings

Proxy

The name or IP address of the SIP server.

Port

The network port the SIP server is listening on. By default this is set to port 5060.

TLSport

If this is other than 0, **TPanel** tries to connect for an encrypted SIP call over this network port. Otherwise it tries to use a random port number. If this is set the standard port number is 5061.

STUN

Sets the IP address for the STUN server.

Domain

Sets the realm for authentication. In most cases this is the same as the *proxy*. Therefore it is optional and mostly left empty.

User

Sets the user name for authentication with the SIP server (proxy address).

Password

Sets the user password so **TPanel** can connect to the SIP server (SIP proxy server).

Network

This defines the type of network to use. If **IPv4** is checked, then this network protocol is used.

If **IPv6** is checked then this protocol will be used. This is also the default protocol and will be used if both protocols are checked. It means also, that the device must be able to use **IPv6** and the network it is connected to must also be able to handle this protocol. In doubt disable **IPv6**.

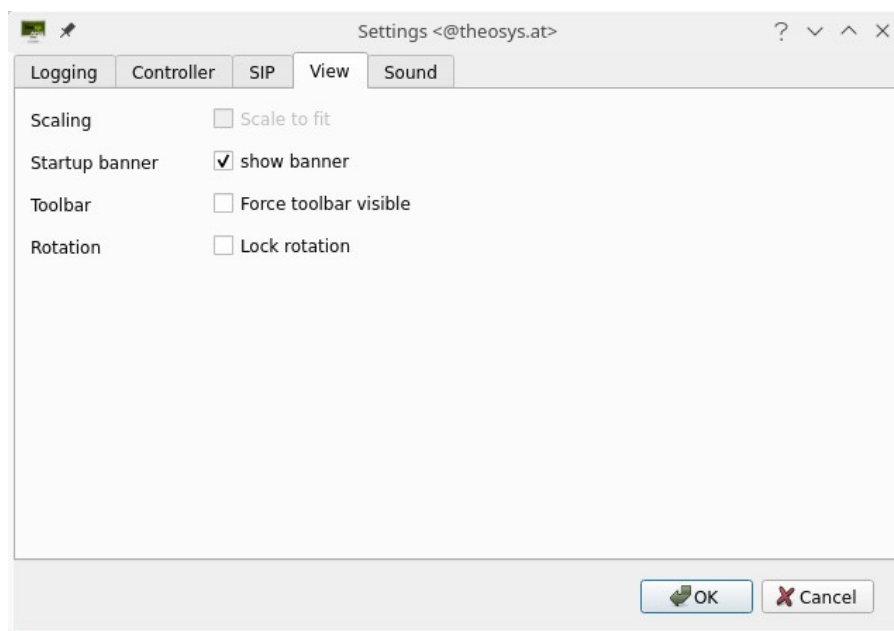
The *Combobox* let you choose the firewall type between your device and the SIP server, if any. By default no firewall is assumed. If there is a firewall and if you have problems to connect to the SIP server, you can choose between to use a *STUN server*, *ICE* or *UPNP*. Ask your local network administrator to find out the correct settings.

State

If this is checked, the SIP settings are enabled and TPanel tries to connect to the SIP proxy server.

View

This tab allows you to set some features of visibility. It allows to select scaling or to force a toolbar to be displayed.



Picture 6: Optical settings

Scaling

On a desktop this is disabled by default and if enabled has no effect.

On a mobile device this is enabled by default and makes sure that the surface fits the size of the display. **TPanel** maintains the aspect ratio which means that you may have a black bar on the left side or at bottom. It depends on the size of the display. If scaling is disabled the real size is used. It depends on the size of the simulated panel (NXD-700Vi has 800 x 480 pixels) and the number of pixels the mobile device offers. For this example we can assume that a mobile device has a higher resolution and therefor the surface would look very small.

Startup banner

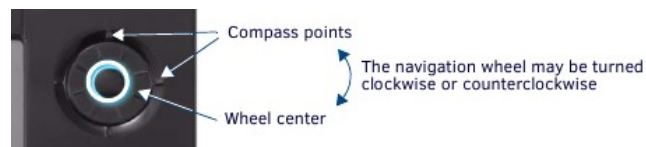
This makes sense only on a desktop. Therefore it is disabled on a mobile device.

If this is checked, TPanel shows a small message on startup from the command line.

```
$ tpanel -c tpanel.cfg
tpanel v1.3.0
(C) Andreas Theofilu <andreas@theosys.at>
This program is under the terms of GPL version 3
```

Toolbar

The toolbar simulates some hard buttons of a real panel. If we take the panel type MVP-5200i for example, we have a round wheel on the right which is also 4 buttons. There is an additional button in the center of the wheel. This buttons are programmable with TPDesign4.



Picture 7: Navigation wheel on an MVP-5200i

Because such buttons are a practical shortcut for navigation, **TPanel** has a toolbar with some similar functions.



Picture 8: Toolbar of TPanel

As you can see, there are navigation buttons into 4 directions and a select button. Then the toolbar offers 2 buttons to control volume.

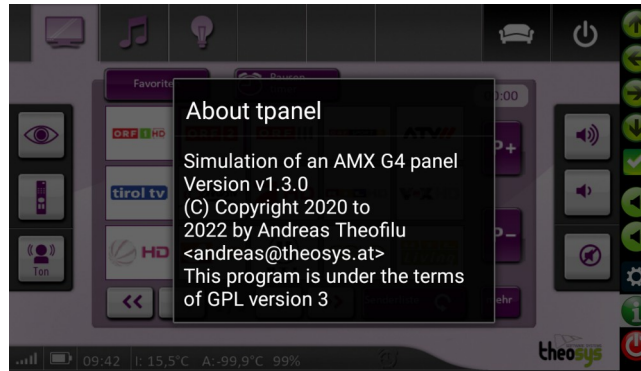
Under the volume buttons you find 3 more buttons:



This opens the settings dialog (look at Setup Dialog at page 14).



This opens an about dialog:



Picture 9: About dialog on a mobile device



This ends the program. If you press this button the application ends truly.

Rotation

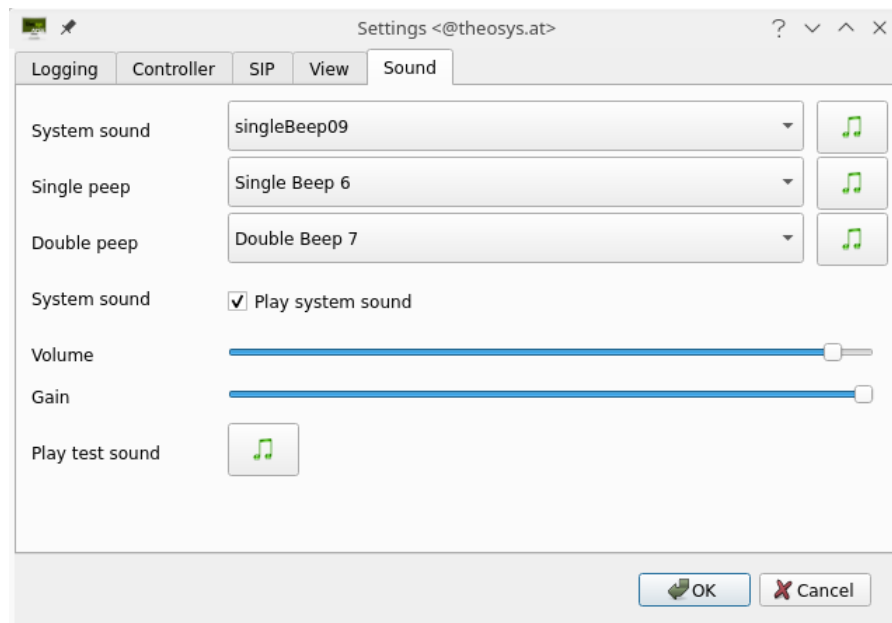
If the box called *Lock rotation* is checked, then the screen is locked and will not rotate anymore.

If this is not checked, the surface will rotate without changing the orientation. This means: If the surface was made for landscape, for example, it will only rotate to normal landscape (landscape left) or inverse landscape (landscape right). It stays at one of this two landscape formats even if it is turned to portrait or inverse portrait.

This setting is independent of any system setting. Even if the system is set to lock screen, the surface will rotate when this is not checked.

Sound

This tab allows the setting of some sound oriented options. It is possible to select a touch sound, the beep sound and the double beep sound.



Picture 10: Sound settings (desktop)

System sound

This noise is played whenever a button was hit. It can be deactivated by removing the check. The drop down box offers some different noises. You can select one and by pressing the button with the green note on the right end of the selector the sound is played.

Single beep

This lets you select the sound of the single beep. This sound is played when **TPanel** receives either the command **BEEP** or **ABEEP**. It is possible to play the sound immediately by pressing on the button on the right side of the selector.

Double beep

This lets you select the sound of the double beep. This sound is played when **TPanel** receives the command **DBEEP** or **ADBEEP**. It is possible to play the sound immediately by pressing on the button on the right side of the selector.

System sound (play system sound)

If this is checked a sound is played on every button hit. The commands **BEEP** and **DBEEP** also play a sound.

If this is not checked, the button hits as well as the commands **BEEP** and **DBEEP** are silent. The commands **ABEEP** and **ADBEEP** are playing always a sound independently of the setting of this check box.

Volume

This slider sets the volume from **TPanel** only. The effective volume depends of the system (global) volume setting and the position of this slider. This slider does not change the system volume setting.

Gain

This defines the sensitivity of the microphone for **TPanel** only.

Play test sound

This button plays a short melody to test the sound settings.

Using TPanel on a mobile device

TPanel is currently not available in the *Play Store* of *Android*. Therefore it must be downloaded from my page at <https://www.theosys.at>. Get the latest version (tpanel_android_vX.X.X.apk) and copy it to a location on your mobile device. The device must run with **Android 11** or newer to be able to use the application.

Enable APK installs on non Samsung devices

1. Go to your phones **Settings**
2. Go to **Security & privacy > More settings**.
3. Tap on **Install apps from external sources**.
4. Select the browser (e.g., Chrome or Firefox) you want to download the APK files from.
5. Toggle **Allow app installs** on.

Enable APK installs on Samsung devices

1. Go to your phone's **Settings**.
2. Go to **Biometrics and security > Install unknown apps**.
3. Select the browser (e.g., Chrome or Firefox) you want to download the APK files from.
4. Toggle **Allow app installs** on.

TPanel on iOS

Because of the restrictions from Apple it is not possible to offer a binary package for iOS. If you want to have this application on your iPhone or iPad, you must compile and install it yourself. You'll find the source and a description of how to do this at [Github](#).

Starting TPanel

Once the application is installed, it can be started. On the first start it presents a small screen with some buttons. Press either on Setup, make a *pinch* gesture or push the *settings button* of the toolbar on the right, if present, to get the settings dialog. Look at Setup Dialog on page 14 for detailed information.

If the setup is completed and the dialog is closed, it takes up to 30 seconds until **TPanel** connects to the NetLinX. Now two scenarios may happen:

- **There is no TP4/TP5 file on the NetLinX.**
After the restart the previous surface (the green one) reappears. Open *TPDesign4/5* and upload your surface to **TPanel** as you would do for a real panel. The transfer display occurs and shows the progress. At the moment the transfer finished, it takes up to 30 seconds until the new interface appears. Now you can use **TPanel** the same way as a real panel.

- **TP4/TP5 file on the NetLinx.**

Upload a TP4/TP5 file to the NetLinx via FTP. Open the setup dialog and enter the name of the file under the tab **Controller** in the field **TP4/TP5 file name**. Make sure the FTP user name and the FTP password is correct. Press the button **Ok** and wait. You'll see a busy dialog during the download and then the application restarts. When it reappears the new surface is visible.

In case of problems

TPanel is far from complete currently. Any command not documented here is not supported or may not work as expected. Therefore it may be that your *NetLinx* program sends commands who are ignored but are mandatory for your surface to work. It is also possible that some commands documented here are not working as expected. *In such cases please inform me!*

Please send an eMail to andreas@theosys.at and attach a short demo program together with a surface file which triggers the error or problem. I will try to fix **TPanel** as fast as possible. In your eMail put the following topics:

- What have I done
- What was expected to happen
- What happened instead

Things who are not working

The following things will not work in the near future because they are proprietary or a secret of AMX. I'm not able to find out without help.

Passwords in resources

If you've defined a password for a resource, the access to a camera for example, the password is encrypted. Until now I was not able to find out the algorithm used to decrypt it. Therefore this will not work. A workaround could be to open the XML file `prj.xma` and search for the section `resourceList`. There you can find the definitions for the wanted resource. The section may look like:

```
<resource>
  <name>Camera 1</name>
  <protocol>HTTP</protocol>
  <user>user</user>
  <password encrypted="1">2312F21D0E2BA367</password>
  <host>8.8.8.8</host>
  <file>snapshot.cgi</file>
  <refresh>1</refresh>
</resource>
```

Change the line

```
<password encrypted="1">2312F21D0E2BA367</password>
```

into

```
<password encrypted="0">password</password>
```

Panel to panel communication

The codec used for communication between panels is proprietary. It is close to a standard but some mandatory parameters are different. I had not the time to investigate in this and my knowledge of this stuff is limited. Therefore this is not working from **TPanel** to a real panel and vice versa.

However: It is implemented and communication between TPanel instances running on any device are working. Connecting to a real AMX panel will work but you'll hear only distorted noise.

TakeNotes

I was not able to find out what it is. Because of this it is not supported.

Remote computer control

I plan to implement this for Linux desktops. Because I for myself don't need it, it has very low priority. So please be patient.

Page commands

Page Commands	
@AFP ^AFP	Flip to specified page (using the named animation). Syntax: <code>''^AFP-<page name>,<animation>,<origin>,<duration>''</code> Variables: - Page Name: If the page name is blank, flip to the previous page - Animation: If blank/invalid, the default animation is fade. - Origin: 1 - 5 - Duration: Transition time in 10ths of a second. Range is 3-30 with 15 (1.5 seconds) as the default Note: <i>The animation is currently not implemented. The page will be visible at once and therefor the parameters Animation, Origin and Duration are ignored!</i> Examples: <code>SEND_COMMAND Panel, ''^AFP-NextPage,slide,4,5''</code> Flip to NextPage sliding from the left for half a second. <code>SEND_COMMAND Panel, ''^AFP-,centerdoorfade,2,10''</code> Flip to NextPage center door fade from the top for a second.
@APG	Add a specific popup page to a specified popup group if it does not already exist. If the new popup is added to a group which has a popup displayed on the current page along with the new pop-up, the displayed popup will be hidden and the new popup will be displayed. Syntax: <code>''@APG-<popup page name>;<popup group name>''</code> Variable: - popup page name = 1 - 50 ASCII characters. Name of the popup page. - popup group name = 1 - 50 ASCII characters. Name of the popup group. Example: <code>SEND_COMMAND Panel, ''@APG-Popup1;Group1''</code> Adds the popup page 'Popup1' to the popup group 'Group1'.
@CPG	Clear all popup pages from specified popup group. Syntax: <code>''@CPG-<popup group name>''</code> Variable: popup group name = 1 - 50 ASCII characters. Name of the popup group. Example: <code>SEND_COMMAND Panel, ''@CPG-Group1''</code> Clears all popup pages from the popup group 'Group1'.
@DPG	Delete a specific popup page from specified popup group if it exists. Syntax: <code>''@DPG-<popup page name>;<popup group name>''</code> Variable: - popup page name = 1 - 50 ASCII characters. Name of the popup page. - popup group name = 1 - 50 ASCII characters. Name of the popup group. Example: <code>SEND_COMMAND Panel, ''@DPG-Popup1;Group1''</code> Deletes the popup page 'Popup1' from the popup group 'Group1'.

Page Commands	
^PCL	Collapse Collapsible Popup Command - Moves the named closeable popup to the collapsed position. Syntax: <code>""^PCL-<popup name>;[optional target page]''</code> Variables: • <i>Popup name</i>: the name of the popup to collapse • <i>Target page</i>: name of the page hosting the popup to affect the change upon. If target page is not specified, the command is applied to the current page. Examples: <code>SEND_COMMAND Panel, ""^PCL-Contacts''</code> Collapse the Contacts popup on the current page. <code>SEND_COMMAND Panel, ""^PCL-Contacts;Teleconference Control''</code> Collapse the Contacts popup on the Teleconference Control pages
^PCT	Collapsible Popup Custom Toggle Command - This is an advanced “toggle” command for collapsible popups, working with a comma-separated list of commands. This list is parsed and a command table is created. Based on the current state of the collapsible popup, the correct command is executed. Note: The previously parsed list is saved and is only parsed again if the command string differs for this popup. Syntax: <code>""^PCT-<popup>,<custom toggle commands>;[optional target page]''</code> Variables: • <i>Popup</i>: popup name • <i>Custom toggle commands</i>: a comma separated list of commands. This list is parsed and a command table is created. The state letters are as follows: o - open c - collapsed d - dynamic, followed by an integer indicating the offset. • <i>*</i> - wildcard, always last in the list Before and after states are separated by -> characters. • <i>Target page</i>: name of the page hosting the popup to affect the change upon. If target page is not specified, the command is applied to the current page. Example: <code>SEND_COMMAND Panel, ""^PCT-RightSlider,c->o,o->d100,*->c''</code> The popup named RightSlider opens if collapsed, move to d100 if open, and collapse otherwise.
@PHE	Set the hide effect for the specified popup page to the named hide effect. Syntax: <code>""@PHE-<popup page name>;<hide effect name>''</code> Variable: • popup page name = 1 - 50 ASCII characters. Name of the page the popup is displayed On. • hide effect name = Refers to the popup effect names being used. Example: <code>SEND_COMMAND Panel, ""@PHE-Popup1;Slide to Left''</code> Sets the Popup1 hide effect name to 'Slide to Left'.

Page Commands	
@PHP	Set the hide effect position. Only 1 coordinate is ever needed for an effect; however, the command will specify both. This command sets the location at which the effect will end at. Syntax: <code>''@PHP-<popup page name>;<x coordinate>,<y coordinate>''</code> Variable: popup page name = 1 - 50 ASCII characters. Name of the page the popup is displayed On. Example: <code>SEND_COMMAND Panel, ''@PHP-Popup1;75,0''</code> Sets the Popup1 hide effect x-coordinate value to 75 and the y-coordinate value to 0.
@PHT	Set the hide effect time for the specified popup page. Syntax: <code>''@PHT-<popup page name>;<hide effect time>''</code> Variable: - popup page name = 1 - 50 ASCII characters. Name of the page the popup is displayed On. - hide effect time = Given in 1/10ths of a second. Example: <code>SEND_COMMAND Panel, ''@PHT-Popup1;50''</code> Sets the Popup1 hide effect time to 5 seconds.
^POP	Open Collapsible Popup Command - Moves the named collapsible popup to the open position. Syntax: <code>""^POP-<popup>;[optional target page]''</code> Variables: <i>Popup</i>: the name of the popup to collapse <i>Target page</i>: name of the page hosting the popup to affect the change upon. If target page is not specified, the command is applied to the current page. Example: <code>SEND_COMMAND Panel, ""^POP-Contacts''</code> Open the Contacts popup on the current page. <code>SEND_COMMAND Panel, ""^POP-Contacts;Teleconference Control''</code> Open the Contacts popup on the Teleconference Control page.
@PPA ^PPA	Close all popups on a specified page. If the page name is empty, the current page is used. Same as the 'Clear Page' command in TPDesign4. Syntax: <code>''@PPA-<page name>''</code> Variable: page name = 1 - 50 ASCII characters. Name of the page the popup is displayed On. Example: <code>SEND_COMMAND Panel, ''@PPA-Page1''</code> Close all pop-ups on Page1.

Page Commands	
@PPF ^PPF	Deactivate a specific popup page on either a specified page or the current page. If the page name is empty, the current page is used (see example 2). If the popup page is part of a group, the whole group is deactivated. This command works in the same way as the 'Hide Popup' command in TPDesign4. Syntax: <code>""@PPF-<popup page name>;<page name>""</code> Variable: - popup page name = 1 - 50 ASCII characters. Name of the popup page. - page name = 1 - 50 ASCII characters. Name of the page the popup is displayed On. Example: <code>SEND_COMMAND Panel, ""@PPF-Popup1;Main""</code> Example 2: <code>SEND_COMMAND Panel, ""@PPF-Popup1""</code> Deactivates the popup page 'Popup1' on the current page.
@PPG ^PPG	Toggle a specific popup page on either a specified page or the current page. If the page name is empty, the current page is used (see example 2). Toggling refers to the activating/deactivating (On/Off) of a popup page. This command works in the same way as the 'Toggle Popup' command in TPDesign4. Syntax: <code>""@PPG-<popup page name>;<page name>""</code> Variable: - popup page name = 1 - 50 ASCII characters. Name of the popup page. - page name = 1 - 50 ASCII characters. Name of the page the popup is displayed On. Example: <code>SEND_COMMAND Panel, ""@PPG-Popup1;Main""</code> Toggles the popup page 'Popup1' on the 'Main' page from one state to another (On/Off). Example 2: <code>SEND_COMMAND Panel, ""@PPG-Popup1""</code> Toggles the popup page 'Popup1' on the current page from one state to another (On/Off).
@PPK ^PPK	Kill refers to the deactivating (Off) of a popup window from all pages. If the pop-up page is part of a group, the whole group is deactivated. This command works in the same way as the 'Clear Group' command in TPDesign 4. Syntax: <code>""@PPK-<popup page name>""</code> Variable: popup page name = 1 - 50 ASCII characters. Name of the popup page. Example: <code>SEND_COMMAND Panel, ""@PPK-Popup1""</code> Kills the popup page 'Popup1' on all pages.

Page Commands	
@PPM ^PPM	Set the modality of a specific popup page to Modal or NonModal. A Modal popup page, when active, only allows you to use the buttons and features on that popup page. All other buttons on the panel page are inactivated. Syntax: "'@PPM-<popup page name>;<mode>'" Variable: - popup page name = 1 - 50 ASCII characters. Name of the popup page. - mode = - NONMODAL converts a previously Modal popup page to a NonModal. - MODAL converts a previously NonModal popup page to Modal. - modal = 1 and non-modal = 0 Example: SEND_COMMAND Panel, "'@PPM-Popup1;Modal'" Sets the popup page 'Popup1' to Modal. SEND_COMMAND Panel, "'@PPM-Popup1;1'" Sets the popup page 'Popup1' to Modal.
@PPN ^PPN	Activate a specific popup page to launch on either a specified page or the current page. If the page name is empty, the current page is used (see example 2). If the popup page is already on, do not re-draw it. This command works in the same way as the 'Show Popup' command in TPDesign4. Syntax: "'@PPN-<popup page name>;<page name>'" Variable: - popup page name = 1 - 50 ASCII characters. Name of the popup page. - page name = 1 - 50 ASCII characters. Name of the page the popup is displayed On. Example: SEND_COMMAND Panel, "'@PPN-Popup1;Main'" Activates 'Popup1' on the 'Main' page. Example 2: SEND_COMMAND Panel, "'@PPN-Popup1'" Activates the popup page 'Popup1' on the current page.
@PPT ^PPT	Set a specific popup page to timeout within a specified time. If timeout is empty, popup page will clear the timeout. Syntax: "'@PPT-<popup page name>;<timeout>'" Variable: - popup page name = 1 - 50 ASCII characters. Name of the popup page. - timeout = Timeout duration in 1/10ths of a second. Example: SEND_COMMAND Panel, "'@PPT-Popup1;30'" Sets the popup page 'Popup1' to timeout within 3 seconds.
@PPX ^PPX	Close all popups on all pages. This command works in the same way as the 'Clear All' command in TPDesign 4. Syntax: "'@PPX'" Example: SEND_COMMAND Panel, "'@PPX'" Close all popups on all pages.

Page Commands	
^PTC	Toggle Collapsible Popup Collapsed Command - Toggles the named collapsible popup between the open and collapsed positions. More specifically, if the popup is not fully collapsed, it is collapsed. Syntax: <code>""^PTC-<popup>;[optional target page]''</code> Variables: • <i>Popup</i>: the name of the popup to toggle • <i>Target page</i>: name of the page hosting the popup to affect the change upon. If target page is not specified, the command is applied to the current page. Examples: <code>SEND_COMMAND Panel, ""^PTC-Contacts''</code> Toggle the Contacts popup collapsed on the current page. <code>SEND_COMMAND Panel, ""^PTC-Contacts;Teleconference Control''</code> Toggle the Contacts popup collapsed on the Teleconference Control page. Note: Collapsible popup send commands do not automatically show the popup on the target page. The popup must be first shown with a standard show command. This applies even when the collapsible popup is a member of a popup group. For all of these commands, if the target page is blank, the current page is used. If the named popup is not collapsible, the commands are ignored.
^PTO	Toggle Collapsed Popup Open Command - Toggles the named collapsible popup between the open and collapsed positions. More specifically, if the popup is not fully open, it is opened. Syntax: <code>""^PTO-<popup>;[optional target page]''</code> Variables: • <i>Popup</i>: the name of the popup to toggle • <i>Target page</i>: name of the page hosting the popup to affect the change upon. If target page is not specified, the command is applied to the current page. Examples: <code>SEND_COMMAND Panel, '^PTO-Contacts'</code> Toggle the Contacts popup open on the current page. <code>SEND_COMMAND Panel, '^PTO-Contacts;Teleconference Control'</code> Toggle the Contacts popup open on the Teleconference Control page. Note: Collapsible popup send commands do not automatically show the popup on the target page. The popup must be first shown with a standard show command. This applies even when the collapsible popup is a member of a popup group. For all of these commands, if the target page is blank, the current page is used. If the named popup is not collapsible, the commands are ignored.

Page Commands	
@PSE	Set the show effect for the specified popup page to the named show effect. Syntax: <code>''@PSE-<popup page name>;<show effect name>''</code> Variable: - popup page name = 1 - 50 ASCII characters. Name of the page the popup is displayed On. - show effect name = Refers to the popup effect name being used. Example: <code>SEND_COMMAND Panel, ''@PSE-Popup1;Slide from Left''</code> Sets the Popup1 show effect name to 'Slide from Left'.
@PSP	Set the show effect position. Only 1 coordinate is ever needed for an effect; however, the command will specify both. This command sets the location at which the effect will begin. Syntax: <code>''@PSP-<popup page name>;<x coordinate>,<y coordinate>''</code> Variable: popup page name = 1 - 50 ASCII characters. Name of the page the popup is displayed On. Example: <code>SEND_COMMAND Panel, ''@PSP-Popup1;100,0''</code> Sets the Popup1 show effect x-coordinate value to 100 and the y-coordinate value to 0.
@PST	Set the show effect time for the specified popup page. Syntax: <code>''@PST-<popup page name>;<show effect time>''</code> Variable: - popup page name = 1 - 50 ASCII characters. Name of the page the popup is displayed On. - show effect time = Given in 1/10ths of a second. Example: <code>SEND_COMMAND Panel, ''@PST-Popup1;50''</code> Sets the Popup1 show effect time to 5 seconds.
PAGE ^PGE	Flips to a page with a specified page name. If the page is currently active, it will not redraw the page. Syntax: <code>''PAGE-<page name>''</code> Variable: page name = 1 - 50 ASCII characters. Name of the page the popup is displayed On. Example: <code>SEND_COMMAND Panel, ''PAGE-Page1''</code> Flips to page1.

Page Commands	
PPOF	Deactivate a specific popup page on either a specified page or the current page. If the page name is empty, the current page is used (see example 2). If the popup page is part of a group, the whole group is deactivated. This command works in the same way as the 'Hide Popup' command in TPDesign4. Syntax: <code>''PPOF-<popup page name>;<page name>''</code> Variable: - popup page name = 1 - 50 ASCII characters. Name of the popup page. - page name = 1 - 50 ASCII characters. Name of the page the popup is displayed On. Example: <code>SEND_COMMAND Panel, ''PPOF-Popup1;Main''</code> Deactivates the popup page 'Popup1' on the Main page. Example 2: <code>SEND_COMMAND Panel, ''PPOF-Popup1''</code> Deactivates the popup page 'Popup1' on the current page.
PPOG	Toggle a specific popup page on either a specified page or the current page. If the page name is empty, the current page is used (see example 2). Toggling refers to the activating/deactivating (On/Off) of a popup page. This command works in the same way as the 'Toggle Popup' command in TPDesign4. Syntax: <code>''PPOG-<popup page name>;<page name>''</code> Variable: - popup page name = 1 - 50 ASCII characters. Name of the popup page. - page name = 1 - 50 ASCII characters. Name of the page the popup is displayed On. Example: <code>SEND_COMMAND Panel, ''PPOG-Popup1;Main''</code> Toggles the popup page 'Popup1' on the Main page from one state to another (On/Off). Example 2: <code>SEND_COMMAND Panel, ''PPOG-Popup1''</code> Toggles the popup page 'Popup1' on the current page from one state to another (On/Off).
PPON	Activate a specific popup page to launch on either a specified page or the current page. If the page name is empty, the current page is used (see example 2). If the popup page is already On, do not re-draw it. This command works in the same way as the 'Show Popup' command in TPDesign4. Syntax: <code>''PPON-<popup page name>;<page name>''</code> Variable: - popup page name = 1 - 50 ASCII characters. Name of the popup page. - page name = 1 - 50 ASCII characters. Name of the page the popup is displayed On. Example: <code>SEND_COMMAND Panel, ''PPON-Popup1; Main''</code> Activates the popup page 'Popup1' on the Main page. Example 2: <code>SEND_COMMAND Panel, ''PPON-Popup1''</code> Activates the popup page 'Popup1' on the current page.

Programming Numbers

The following information provides the programming numbers for colors, fonts, and borders.

Colors can be used to set the colors on buttons, sliders, and pages. The lowest color number represents the lightest color-specific display; the highest number represents the darkest display. For example, 0 represents light red, and 5 is dark red.

RGB Triplets and Names For Basic 88 Colors

RGB Values for all 88 Basic Colors									
Index No.	Name	Red	Green	Blue	Index No.	Name	Red	Green	Blue
0	Very Light Red	255	0	0	45	Medium Aqua	0	80	159
1	Light Red	223	0	0	46	Dark Aqua	0	64	127
2	Red	191	0	0	47	Very Dark Aqua	0	48	95
3	Medium Red	159	0	0	48	Very Light Blue	0	0	255
4	Dark Red	127	0	0	49	Light Blue	0	0	223
5	Very Dark Red	95	0	0	50	Blue	0	0	191
6	Very Light Orange	255	128	0	51	Medium Blue	0	0	159
7	Light Orange	223	112	0	52	Dark Blue	0	0	127
8	Orange	191	96	0	53	Very Dark Blue	0	0	95
9	Medium Orange	159	80	0	54	Very Light Purple	128	0	255
10	Dark Orange	127	64	0	55	Light Purple	112	0	223
11	Very Dark Orange	95	48	0	56	Purple	96	0	191
12	Very Light Yellow	255	255	0	57	Medium Purple	80	0	159
13	Light Yellow	223	223	0	58	Dark Purple	64	0	127
14	Yellow	191	191	0	59	Very Dark Purple	48	0	95
15	Medium Yellow	159	159	0	60	Very Light Magenta	255	0	255
16	Dark Yellow	127	127	0	61	Light Magenta	223	0	223
17	Very Dark Yellow	95	95	0	62	Magenta	191	0	191
18	Very Light Lime	128	255	0	63	Medium Magenta	159	0	159
19	Light Lime	112	223	0	64	Dark Magenta	127	0	127
20	Lime	96	191	0	65	Very Dark Magenta	95	0	95
21	Medium Lime	80	159	0	66	Very Light Pink	255	0	128
22	Dark Lime	64	127	0	67	Light Pink	223	0	112
23	Very Dark Lime	48	95	0	68	Pink	191	0	96
24	Very Light Green	0	255	0	69	Medium Pink	159	0	80
25	Light Green	0	223	0	70	Dark Pink	127	0	64
26	Green	0	191	0	71	Very Dark Pink	95	0	48
27	Medium Green	0	159	0	72	White	255	255	255
28	Dark Green	0	127	0	73	Grey1	238	238	238
29	Very Dark Green	0	95	0	74	Grey3	204	204	204
30	Very Light Mint	0	255	128	75	Grey5	170	170	170
31	Light Mint	0	223	112	76	Grey7	136	136	136
32	Mint	0	191	96	77	Grey9	102	102	102
33	Medium Mint	0	159	80	78	Grey4	187	187	187
34	Dark Mint	0	127	64	79	Grey6	153	153	153

RGB Values for all 88 Basic Colors									
Index No.	Name	Red	Green	Blue	Index No.	Name	Red	Green	Blue
35	Very Dark Mint	0	95	48	80	Grey8	119	119	119
36	Very Light Cyan	0	255	255	81	Grey10	85	85	85
37	Light Cyan	0	223	223	82	Grey12	51	51	51
38	Cyan	0	191	191	83	Grey13	34	34	34
39	Medium Cyan	0	159	159	84	Grey2	221	221	221
40	Dark Cyan	0	127	127	85	Grey11	68	68	68
41	Very Dark Cyan	0	95	95	86	Grey14	17	17	17
42	Very Light Aqua	0	128	255	87	Black	0	0	0
43	Light Aqua	0	112	223	255	TRANSPARENT	99	53	99
44	Aqua	0	96	161					

Font Styles And ID Numbers

Font styles can be used to program the text fonts on buttons, sliders, and pages. The following chart shows the default font type and their respective ID numbers generated by TPDesign4.

Default Font Styles and ID Numbers					
Font ID #	Font type	Size	Font ID #	Font type	Size
1	Courier New	9	19	Arial	9
2	Courier New	12	20	Arial	10
3	Courier New	18	21	Arial	12
4	Courier New	26	22	Arial	14
5	Courier New	32	23	Arial	16
6	Courier New	18	24	Arial	18
7	Courier New	26	25	Arial	20
8	Courier New	34	26	Arial	24
9	AMX Bold	14	27	Arial	36
10	AMX Bold	20	28	Arial Bold	10
11	AMX Bold	36	29	Arial Bold	8

NOTE: Fonts must be imported into a TPDesign4 project file. The font ID numbers are assigned by TPDesign4. These values are also listed in the Generate Programmer's Report.

Border Styles And Programming Numbers

Border styles can be used to program borders on buttons, sliders, and popup pages.

Border Styles and Programming Numbers			
No.	Border Styles	No.	Border Styles
0 - 1	No border	10 - 11	Picture frame
2	Single line	12	Double line
3	Double line	20	Bevel-S
4	Quad line	21	Bevel-M
5 - 6	Circle 15	22 - 23	Circle 15
7	Single line	24 - 27	Neon inactive-S
8	Double line	40 - 41	Diamond 55
9	Quad line		

Button Query Commands

Button Query commands reply back with a custom event. There will be one custom event for each button/state combination. Each query is assigned a unique custom event type. The following example is for debug purposes only:

```
NetLinx Example: CUSTOM_EVENT[device,Address, Custom event type]
DEFINE_EVENT
    CUSTOM_EVENT[TP,529,1001]    // Text
    CUSTOM_EVENT[TP,529,1002]    // Bitmap
    CUSTOM_EVENT[TP,529,1003]    // Icon
    CUSTOM_EVENT[TP,529,1004]    // Text Justification
    CUSTOM_EVENT[TP,529,1005]    // Bitmap Justification
    CUSTOM_EVENT[TP,529,1006]    // Icon Justification
    CUSTOM_EVENT[TP,529,1007]    // Font
    CUSTOM_EVENT[TP,529,1008]    // Text Effect Name
    CUSTOM_EVENT[TP,529,1009]    // Text Effect Color
    CUSTOM_EVENT[TP,529,1010]    // Word Wrap
    CUSTOM_EVENT[TP,529,1011]    // ON state Border Color
    CUSTOM_EVENT[TP,529,1012]    // ON state Fill Color
    CUSTOM_EVENT[TP,529,1013]    // ON state Text Color
    CUSTOM_EVENT[TP,529,1014]    // Border Name
    CUSTOM_EVENT[TP,529,1015]    // Opacity
{
    Send_String 0, "'ButtonGet Id=',ITOA(CUSTOM.ID), ' Type=',ITOA(CUSTOM.TYPE)"
    Send_String 0, "'Flag   =',ITOA(CUSTOM.FLAG)"
    Send_String 0, "'VALUE1 =',ITOA(CUSTOM.VALUE1)"
    Send_String 0, "'VALUE2 =',ITOA(CUSTOM.VALUE2)"
    Send_String 0, "'VALUE3 =',ITOA(CUSTOM.VALUE3)"
    Send_String 0, "'TEXT   =',CUSTOM.TEXT"
    Send_String 0, "'TEXT LENGTH =',ITOA(LENGTH_STRING(CUSTOM.TEXT))"
}
```

All custom events have the following 7 fields:

Custom Event Fields	
Uint Flag	0 means text is a standard string, 1 means Unicode encoded string
slong value1	button state number
slong value2	actual length of string (this is not encoded size)
slong value3	index of first character (usually 1 or same as optional index)
string text	the text from the button
text length (string encode)	button text length

These fields are populated differently for each query command. The text length (String Encode) field is not used in any command. These Button Commands are used in NetLinx Studio and are case insensitive:

Button Commands	
^ANI	Run a button animation (in 1/10 second). Syntax: <code>''^ANI-<vt addr range>,<start state>,<end state>,<time>''</code> Variable: • variable text address range = 1 - 4000. • start state = Beginning of button state (0= current state). • end state = End of button state. • time = In 1/10 second intervals. Example: <code>SEND_COMMAND Panel, ''^ANI-500,1,25,100''</code> Runs a button animation at text range 500 from state 1 to state 25 for 10 second.
^APF	Add page flip action to a button if it does not already exist. Syntax: <code>''^APF-<vt addr range>,<page flip action>,<page name>''</code> Variable: • variable text address range = 1 - 4000. • page flip action = ◦ Stan[dardPage] - Flip to standard page ◦ Prev[iousPage] - Flip to previous page ◦ Show[Popup] - Show Popup page ◦ Hide[Popup] - Hide Popup page ◦ Togg[lePopup] - Toggle popup state ◦ ClearG[roup] - Clear popup page group from all pages ◦ ClearP[age] - Clear all popup pages from a page with the specified page name ◦ ClearA[ll] - Clear all popup pages from all pages • page name = 1 - 50 ASCII characters. Example: <code>SEND_COMMAND Panel, ''^APF-400,Stan,Main Page''</code> Assigns a button to a standard page flip with page name 'Main Page'.
^BAT	Append non-unicode text. Syntax: <code>''^BAT-<vt addr range>,<button states range>,<new text>''</code> Variable: • variable text address range = 1 - 4000. • button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). • new text = 1 - 50 ASCII characters. Example: <code>SEND_COMMAND Panel, ''^BAT-520,1,Enter City''</code> Appends the text 'Enter City' to the button's OFF state.

Button Commands	
^BAU	Append unicode text. Same format as ^UNI. Syntax: <code>''^BAU-<vt addr range>,<button states range>,<unicode text>''</code> Variable: - variable text address range = 1 - 4000. - button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). - unicode text = 1 - 50 ASCII characters. Unicode characters must be entered in Hex format. Example: <code>SEND_COMMAND Panel, ''^BAU-520,1,00770062''</code> Appends Unicode text '00770062' to the button's OFF state.
^BCB	Set the border color to the specified color. Only if the specified border color is not the same as the current color. <i>Note: Color can be assigned by color name (without spaces), number or R,G,B value (RRGGBB or RRGGBBAA).</i> Syntax: <code>''^BCB-<vt addr range>,<button states range>,<color value>''</code> Variable: - variable text address range = 1 - 4000. - button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). - color value = Refer to the RGB Triplets and Names For Basic 88 Colors table on page 86 for details. Example: <code>SEND_COMMAND Panel, ''^BCB-500.504&510,1,12''</code> Sets the Off state border color to 12 (Yellow). Colors can be set by Color Numbers, Color name, R,G,B,alpha colors (RRGGBBAA) and R, G & B colors values (RRGGBB). Refer to the RGB Triplets and Names For Basic 88 Colors table on page 37.

Button Commands	
?BCB	Get the current border color. Syntax: <code>''?BCB-<vt addr range>,<button states range>''</code> Variable: - variable text address range = 1 - 4000. - button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). - custom event type 1011: - Flag - zero - Value1 - Button state number - Value2 - Actual length of string (should be 9) - Value3 - Zero - Text - Hex encoded color value (ex: #000000FF) - Text length - Color name length (should be 9) Example: SEND_COMMAND Panel, ''?BCB-529,1'' Gets the button 'OFF state' border color. information. The result sent to the Master would be: ButtonGet Id = 529 Type = 1011 Flag = 0 VALUE1 = 1 VALUE2 = 9 VALUE3 = 0 TEXT = #222222FF TEXT LENGTH = 9
^BCF	Set the fill color to the specified color. Only if the specified fill color is not the same as the current color. <i>Note: Color can be assigned by color name (without spaces), number or R,G,B value (RRGGBB or RRGGBBAA).</i> Syntax: <code>''^BCF-<vt addr range>,<button states range>,<color value>''</code> Variable: - variable text address range = 1 - 4000. - button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). - color value = Refer to the RGB Triplets and Names For Basic 88 Colors table on page 37 for details. Example: SEND_COMMAND Panel, ''^BCF-500.504&510.515,1,12'' SEND_COMMAND Panel, ''^BCF-500.504&510.515,1,Yellow'' SEND_COMMAND Panel, ''^BCF-500.504&510.515,1,#F4EC0A63'' SEND_COMMAND Panel, ''^BCF-500.504&510.515,1,#F4EC0A'' Sets the Off state fill color by color number. Colors can be set by Color Numbers, Color name, R,G,B,alpha colors (RRGGBBAA) and R, G & B colors values (RRGGBB).

Button Commands	
?BCF	Get the current fill color. Syntax: <code>''?BCF-<vt addr range>,<button states range>''</code> Variable: - variable text address range = 1 - 4000. - button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). - custom event type 1012: - Flag - Zero - Value1 - Button state number - Value2 - Actual length of string (should be 9) - Value3 - Zero - Text - Hex encoded color value (ex: #000000FF) - Text length - Color name length (should be 9) Example: SEND_COMMAND Panel, ''?BCF-529,1'' Gets the button 'OFF state' fill color information. The result sent to the Master would be: ButtonGet Id = 529 Type = 1012 Flag = 0 VALUE1 = 1 VALUE2 = 9 VALUE3 = 0 TEXT = #FF8000FF TEXT LENGTH = 9
^BCT	Set the text color to the specified color. Only if the specified text color is not the same as the current color. <i>Note: Color can be assigned by color name (without spaces), number or R,G,B value (RRGGBB or RRGGBBAA).</i> Syntax: <code>''^BCT-<vt addr range>,<button states range>,<color value>''</code> Variable: - variable text address range = 1 - 4000. - button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). - color value = Refer to the RGB Triplets and Names For Basic 88 Colors table on page 86 for details. Example: SEND_COMMAND Panel, ''^BCT-500.504&510,1,12'' Sets the Off state border color to 12 (Yellow). Colors can be set by Color Numbers, Color name, R,G,B,alpha colors (RRGGBBAA) and R, G & B colors values (RRGGBB).

Button Commands	
?BCT	Get the current text color. Syntax: <code>''?BCT-<vt addr range>,<button states range>''</code> Variable: - variable text address range = 1 - 4000. - button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). - custom event type 1013: - Flag - Zero - Value1 - Button state number - Value2 - Actual length of string (should be 9) - Value3 - Zero - Text - Hex encoded color value (ex: #000000FF) - Text length - Color name length (should be 9) Example: SEND_COMMAND Panel, ''?BCT-529,1'' Gets the button 'OFF state' text color information. The result sent to Master would be: ButtonGet Id = 529 Type = 1013 Flag = 0 VALUE1 = 1 VALUE2 = 9 VALUE3 = 0 TEXT = #FFFFFFEF TEXT LENGTH = 9
^BDO	Set the button draw order - Determines what order each layer of the button is drawn. Syntax: <code>''^BDO-<vt addr range>,<button states range>,<1-5><1-5><1-5><1-5><1-5>''</code> Variable: - variable text address range = 1 - 4000. - button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). - layer assignments = - Fill Layer = 1 - Image Layer = 2 - Icon Layer = 3 - Text Layer = 4 - Border Layer = 5 <i>Note: The layer assignments are from bottom to top. The default draw order is 12345.</i> Example: SEND_COMMAND Panel, ''^BDO-530,1&2,51432'' Sets the button's variable text 530 ON/OFF state draw order (from bottom to top) to Border, Fill, Text, Icon, and Image. Example 2: SEND_COMMAND Panel, ''^BDO-1,0,12345'' Sets all states of a button back to its default drawing order.

Button Commands	
^BFB	Set the feedback type of the button. ONLY works on General-type buttons. Syntax: <code>''^BFB-<vt addr range>,<feedback type>''</code> Variable: - variable text address range = 1 - 4000. - feedback type = (None, Channel, Invert, On (Always on), Momentary, and Blink). Example: <code>SEND_COMMAND Panel, ''^BFB-500,Momentary''</code> Sets the Feedback type of the button to 'Momentary'.
^BIM	Set the input mask for the specified address. Syntax: <code>''^BIM-<vt addr range>,<input mask>''</code> Variable: - variable text address range = 1 - 4000. - input mask = Refer to the Text Area Input Masking table on page 176 for character types. Example: <code>SEND_COMMAND Panel, ''^BIM-500,AAAAAAAAAA''</code> Sets the input mask to ten 'A' characters, that are required, to either a letter or digit (entry is required).

Button Commands	
^BMC	Button copy command. Copy attributes of the source button to all the destination buttons. Note that the source is a single button state. Each state must be copied as a separate command. The <codes> section represents what attributes will be copied. All codes are 2 char pairs that can be separated by comma, space, percent or just ran together. Syntax: <code>''^BMC-<vt addr range>,<button states range>,<source port>,<source address>,<source state>,<codes>''</code> Variable: • variable text address range = 1 - 4000. • button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). • source port = 1 - 100. • source address = 1 - 4000. • source state = 1 - 256. • codes: - BM – Picture/Bitmap - BR – Border - CB – Border Color - CF – Fill Color - CT – Text Color - EC – Text effect color - EF – Text effect - FT – Font - IC – Icon - JB – Bitmap alignment - JI – Icon alignment - JT – Text alignment - OP – Opacity - SO – Button Sound - TX – Text - VI – Video slot ID - WW – Word wrap on/off Example: <code>SEND_COMMAND Panel, ''^BMC-425,1,1,500,1,BR''</code> or <code>SEND_COMMAND Panel, ''^BMC-425,1,1,500,1,%BR''</code> Copies the OFF state border of button with a variable text address of 500 onto the OFF state border of button with a variable text address of 425. Example 2: <code>SEND_COMMAND Panel, ''^BMC-150,1,1,315,1,%BR%FT%TX%BM%IC%CF%CT''</code> Copies the OFF state border, font, Text, bitmap, icon, fill color and text color of the button with a variable text address of 315 onto the OFF state border, font, Text, bitmap, icon, fill color and text color of the button with a variable text address of 150.
^BMF	Set any/all button parameters by sending embedded codes and data. Syntax: <code>''^BMF-<vt addr range>,<button states range>,<data>''</code> Variables: • variable text address char array = 1 - 4000. • button states range = 1 - 256 for multi-state buttons (0 = All states, for General

Button Commands

- buttons 1 = Off state and 2 = On state).
- level range = 1 - 600 (level value is 1 - 65535).
- data:
 - '%B<border style>' = Set the border style name. See the Border Styles and Programming Numbers table on page 39.
 - '%B',<border 0-27,40,41> = Set the border style number. See the Border Styles and Programming Numbers table on page 39.
 - '%DO<1-5><1-5><1-5><1-5><1-5>' = Set the draw order. Listed from bottom to top. Refer to the ^BDO command for more information.
 - '%F', = Set the font. See the Default Font Styles and ID Numbers table on page 39.
 - '%F' = Set the font. See the Default Font Styles and ID Numbers table on page 39.
 - ~~'%MI<mask image>' = Set the mask image. Refer to the ^BMI command for more information.~~
 - '%R' = Sets button location and also resizes the button. Takes four parameters: <left>, <top>, <right>, <bottom>.
 - '%T<text >' = Set the text using ASCII characters (empty is clear).
 - '%P<bitmap>' = Set the picture/bitmap filename (empty is clear).
 - '%I<icon 01-9900, 0-clear>' = Set the icon using values of 01 - 9900 (icon numbers are assigned in the TPDesign4 Resource Manager tab - Slots section).
 - '%J',<alignment of text 1-9> = As shown the following telephone keypad alignment chart:

0

1	2	3
4	5	6
7	8	9

Zero can be used for an absolute position.

- '%JT<alignment of text 0-9>' = As shown the above telephone keypad alignment chart, BUT the 0 (zero) is absolute and followed by ',<left>,<top>'
 - '%JB<alignment of bitmap/picture 0-9>' = As shown the above telephone keypad alignment chart BUT the 0 (zero) is absolute and followed by ',<left>,<top>'
 - '%JI<alignment of icon 0-9>' = As shown the above telephone keypad alignment chart, BUT the 0 (zero) is absolute and followed by ',<left>,<top>'
- For some of these commands and values, refer to the RGB Triplets and Names For Basic 88 Colors table on page 37.
- '%CF<on fill color>' = Set Fill Color.
 - '%CB<on border color>' = Set Border Color.
 - '%CT<on text color>' = Set Text Color.
 - '%SW<1 or 0>' = Show/hide a button.
 - '%SO<sound>' = Set the button sound.
 - '%EN<1 or 0>' = Enable/disable a button.
 - '%WW<1 or 0>' = Word wrap ON/OFF.
 - '%GH<bargraph hi>' = Set the bargraph upper limit.
 - '%GL<bargraph low>' = Set the bargraph lower limit.
 - '%GN<bargraph slider name>' = Set the bargraph slider name/Joystick cursor name.

Button Commands	
	'%GC<bargraph slider color>' = Set the bargraph slider color/Joystick cursor color. '%OT<feedback type>' = Set the Feedback (Output) Type to one of the following: None, Channel, Invert, ON (Always ON), Momentary, or Blink. '%OP<0-255>' = Set the button opacity to either Invisible (value=0) or Opaque (value=255). '%OP#<00-FF>' = Set the button opacity to either Invisible (value=00) or Opaque (value=FF). '%UN<Unicode text>' = Set the Unicode text. See the ^UNI section for the text format. '%EF<text effect name>' = Set the text effect. '%EC<text effect color>' = Set the text effect color. Example: <pre>SEND_COMMAND Panel, "'^BMF-500,1,%B10%CFRed%CB Blue %CTBlack%Ptest.png'"</pre> Sets the button OFF state as well as the Border, Fill Color, Border Color, Text Color, and Bitmap.
^BML	Set the maximum length of the text area button. If this value is set to zero (0), the text area has no max length. The maximum length available is 2000. This is only for a Text area input button and not for a Text area input masking button. Syntax: <pre>"'^BML-<vt addr range>,<max length>'"</pre> Variable: • variable text address range = 1 - 4000. • max length = 2000 (0=no max length). Example: <pre>SEND_COMMAND Panel, "'^BML-500,20'"</pre> Sets the maximum length of the text area input button to 20 characters.

Button Commands	
^BMP	Assign a picture to those buttons with a defined address range. Syntax: <code>''^BMP-<vt addr range>,<button states range>,<name of bitmap/picture>,[bitmap index],[optional justification]''</code> Variable: • variable text address range = 1 - 4000. • button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). • name of bitmap/picture = 1 - 50 ASCII characters. • Optional bitmap index = 0 - 5, the state bitmap index to assign the bitmap. The indexes are defined as: ◦ 0 - Chameleon Image (if present) ◦ 1 - Bitmap 1 (Bitmap) ◦ 2 - Bitmap 2 (Icon) ◦ 3 - Bitmap 3 (Bitmap) ◦ 4 - Bitmap 4 (Bitmap) ◦ 5 - Bitmap 5 (Bitmap) • Optional justification = 0-10 where: ◦ 0 - Absolute position: If absolute justification is set, the next two parameters are the X and Y offset of the bitmap for the referenced index. ◦ 1 - top left ◦ 2 - top center ◦ 3 - top right ◦ 4 - middle left ◦ 5 - middle center ◦ 6 - middle right ◦ 7 - bottom left ◦ 8 - bottom center ◦ 9 - bottom right ◦ 10 - scale to fit ◦ If no justification is specified, the current justification is used. Example: <code>SEND_COMMAND Panel, ''^BMP-500.504&510.515,1,bitmap.png''</code> Sets the OFF state picture for the buttons with variable text ranges of 500-504 & 510-515.

Button Commands	
?BMP	Get the current bitmap name. Syntax: <code>''?BMP-<vt addr range>,<button states range>''</code> Variable: - variable text address range = 1 - 4000. - button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 custom event type 1002: - Flag - Zero - Value1 - Button state number - Value2 - Actual length of string - Value3 - Zero - Text - String that represents the bitmap name - Text length - Bitmap name text length (should be 9) Example: SEND_COMMAND Panel, ''?BMP-529,1'' Gets the button 'OFF state' bitmap information. The result sent to the Master would be: ButtonGet Id = 529 Type = 1002 Flag = 0 VALUE1 = 1 VALUE2 = 9 VALUE3 = 0 TEXT = Buggs.png TEXT LENGTH = 9
^BOP	Set the button opacity. The button opacity can be specified as a decimal between 0 - 255, where zero (0) is invisible and 255 is opaque, or as a HEX code, as used in the color commands by preceding the HEX code with the # sign. In this case, #00 becomes invisible and #FF becomes opaque. If the opacity is set to zero (0), this does not make the button inactive, only invisible. Syntax: <code>''^BOP-<vt addr range>,<button states range>,<button opacity>''</code> Variable: - variable text address range = 1 - 4000. - button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). - button opacity = 0 (invisible) - 255 (opaque). Example: SEND_COMMAND Panel, ''^BOP-500.504&510.515,1,200'' SEND_COMMAND Panel, ''^BOP-500.504&510.515,1,#C8'' Both examples set the opacity of the buttons with the variable text range of 500-504 and 510-515 to 200.

Button Commands	
?BOP	Get the overall button opacity. Syntax: <code>''?BOP-<vt addr range>,<button states range>''</code> Variable: • variable text address range = 1 - 4000. • button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). • custom event type 1015: • Flag - Zero • Value1 - Button state number • Value2 - Opacity • Value3 - Zero • Text - Blank • Text length - Zero Example: <code>SEND_COMMAND Panel, ''?BOP-529,1''</code> Gets the button 'OFF state' opacity information. The result sent to the Master would be: ButtonGet Id = 529 Type = 1015 Flag = 0 VALUE1 = 1 VALUE2 = 200 VALUE3 = 0 TEXT = TEXT LENGTH = 0
^BOR	Set a border to a specific border style associated with a border value for those buttons with a defined address range. Refer to the Border Styles and Programming Numbers table on page 39 for more information. Syntax: <code>''^BOR-<vt addr range>,<border style name or border value>''</code> Variable: • variable text address range = 1 - 4000. • border style name = Refer to the Border Styles and Programming Numbers table on page 87. • border value = 0 - 41. Examples: <code>SEND_COMMAND Panel, ''^BOR-500.504&510.515,10''</code> Sets the border by number (#10) to those buttons with the variable text range of 500-504 & 510-515. <code>SEND_COMMAND Panel, ''^BOR-500.504&510,AMX Elite -M''</code> Sets the border by name (AMX Elite) to those buttons with the variable text range of 500-504 & 510-515. The border style is available through the TPDesign4 border-style drop-down list. Refer to the TPD4 Border Styles by Name table on page 39 for more information.

Button Commands	
^BOS	Set the button to display either a Video or Non-Video window. Syntax: <code>''^BOS-<vt addr range>,<button states range>,<video state>''</code> Variable: • variable text address range = 1 - 4000. • button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). • video state = Video Off = 0 and Video On = 1. Example: <code>SEND_COMMAND Panel, ''^BOS-500,1,1''</code> Sets the button to display video.
^BRD	Set the border of a button state/states. Only if the specified border is not the same as the current border. The border names are available through the TPDesign4 border-name drop-down list. Syntax: <code>''^BRD-<vt addr range>,<button states range>,<border name>''</code> Variable: • variable text address range = 1 - 4000. • button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). • border name = Refer to the Border Styles and Programming Numbers table on page 39. Example: <code>SEND_COMMAND Panel, ''^BRD-500.504&510.515,1&2,Quad Line''</code> Sets the border by name (Quad Line) to those buttons with the variable text range of 500-504 & 510-515. Refer to the TPD4 Border Styles by Name table on page 39.

Button Commands	
?BRD	Get the current border name. Syntax: <code>''?BRD-<vt addr range>,<button states range>''</code> Variable: • variable text address range = 1 - 4000. • button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). • custom event type 1014: • Flag - Zero • Value1 - Button state number • Value2 - Actual length of string • Value3 - Zero • Text - String that represents border name • Text length - Border name length Example: <code>SEND_COMMAND Panel, ''?BRD-529,1''</code> Gets the button 'OFF state' border information. The result sent to the Master would be: ButtonGet Id = 529 Type = 1014 Flag = 0 VALUE1 = 1 VALUE2 = 22 VALUE3 = 0 TEXT = Double Bevel Raised -L TEXT LENGTH = 22
^BSM	Submit text for text area buttons. This command causes the text areas to send their text as strings to the NetLinx Master. Syntax: <code>''^BSM-<vt addr range>''</code> Variable: • variable text address range = 1 - 4000. Example: <code>SEND_COMMAND Panel, ''^BSM-500''</code> Submits the text of the text area button.
^BS0	Set the sound played when a button is pressed. If the sound name is blank the sound is then cleared. If the sound name is not matched, the button sound is not changed. Syntax: <code>''^BS0-<vt addr range>,<button states range>,<sound name>''</code> Variable: • variable text address range = 1 - 4000. • button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). • sound name = (blank - sound cleared, not matched - button sound not changed). Example: <code>SEND_COMMAND Panel, ''^BS0-500,1&2,music.wav''</code> Assigns the sound 'music.wav' to the button Off/On states.

Button Commands	
^BSP	Set the button size and its position on the page. Syntax: <code>''^BSP-<vt addr range>,<left>,<top>,<right>,<bottom>''</code> Variable: • variable text address range = 1 - 4000. • left = left side of page. • top = top of page. • right = right side of page. • bottom = bottom of page. Example: <code>SEND_COMMAND Panel, ''^BSP-530, left, top''</code> Sets the button with variable text 530 in the left side top of page.
^BWW	Set the button word wrap feature to those buttons with a defined address range. By default, word-wrap is Off. Syntax: <code>''^BWW-<vt addr range>,<button states range>,<word wrap>''</code> Variable: • variable text address range = 1 - 4000. • button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). • word wrap = (0=Off and 1=On). Default is Off. Example: <code>SEND_COMMAND Panel, ''^BWW-500,1,1''</code> Sets the word wrap on for the button's Off state.
?BWW	Get the current word wrap flag status. Syntax: <code>''?BWW-<vt addr range>,<button states range>''</code> Variable: • variable text address range = 1 - 4000. • button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). • custom event type 1010: • Flag - Zero • Value1 - Button state number • Value2 - 0 = no word wrap, 1 = word wrap • Value3 - Zero • Text - Blank • Text length - Zero Example: <code>SEND_COMMAND Panel, ''?BWW-529,1''</code> Gets the button 'OFF state' word wrap flag status information. The result sent to the Master would be: ButtonGet Id = 529 Type = 1010 Flag = 0 VALUE1 = 1 VALUE2 = 1 VALUE3 = 0 TEXT = TEXT LENGTH = 0

Button Commands	
^CPF	Clear all page flips from a button. Syntax: <code>''^CPF-<vt addr range>''</code> Variable: variable text address range = 1 - 4000. Example: <code>SEND_COMMAND Panel, ''^CPF-500''</code> Clears all page flips from the button.
^DPF	Delete page flips from button if it already exists. Syntax: <code>''^DPF-<vt addr range>,<actions>,<page name>''</code> Variable: - variable text address range = 1 - 4000. - actions = Stan[dardPage] - Flip to standard page Prev[iousPage] - Flip to previous page Show[Popup] - Show Popup page Hide[Popup] - Hide Popup page Togg[lePopup] - Toggle popup state ClearG[roup] - Clear popup page group from all pages ClearP[age] - Clear all popup pages from a page with the specified page name ClearA[ll] - Clear all popup pages from all pages - page name = 1 - 50 ASCII characters. Example: <code>SEND_COMMAND Panel, ''^DPF-409,Prev''</code> Deletes the assignment of a button from flipping to a previous page.
^ENA	Enable or disable buttons with a set variable text range. Syntax: <code>''^ENA-<vt addr range>,<command value>''</code> Variable: - variable text address range = 1 - 4000. - command value = (0= disable, 1= enable) Example: <code>SEND_COMMAND Panel, ''^ENA-500.504&510.515,0''</code> Disables button pushes on buttons with variable text range 500-504 & 510-515.

Button Commands	
^FON	Set a font to a specific Font ID value for those buttons with a defined address range. Font ID numbers are generated by the TPDesign4 programmers report. Syntax: <code>''^FON-<vt addr range>,<button states range>,''</code> Variable: - variable text address range = 1 - 4000. - button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). - font value = range = 1 - XXX. Refer to the Default Font Styles and ID Numbers section on page 87. Example: <code>SEND_COMMAND Panel, ''^FON-500.504&510.515,1&2,4''</code> Sets the font size to font ID #4 for the On and Off states of buttons with the variable text range of 500-504 & 510-515. <i>Note: The Font ID is generated by TPD4 and is located in TPD4 through the Main menu. Panel > Generate Programmer's Report >Text Only Format >Readme.txt.</i>
?FON	Get the current font index. Syntax: <code>''?FON-<vt addr range>,<button states range>''</code> Variable: - variable text address range = 1 - 4000. - button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). - custom event type 1007: - Flag - Zero - Value1 - Button state number - Value2 - Font index - Value3 - Zero - Text - Blank - Text length - Zero Example: <code>SEND COMMAND Panel, ''?FON-529,1''</code> Gets the button 'OFF state' font type index information. The result sent to the Master would be: <pre> ButtonGet Id = 529 Type = 1007 Flag = 0 VALUE1 = 1 VALUE2 = 72 VALUE3 = 0 TEXT = TEXT LENGTH = 0 </pre>
^GDI	Change the bargraph drag increment. Syntax: <code>''^GDI-<vt addr range>,<bargraph drag increment>''</code> Variable: - variable text address range = 1 - 4000. - bargraph drag increment = The default drag increment is 256. Example: <code>SEND_COMMAND Panel, ''^GDI-7,128''</code> Sets the bargraph with variable text 7 to a drag increment of 128.

Button Commands	
^GIV	Bargraph invert command - Invert the bargraph to move in the opposite direction. Syntax: <code>''^GIV-<addr range>,<invert=1, non-inverted=0>''</code> Variables: <i>address range:</i> Address codes of buttons to affect. A '.' between addresses includes the range, and & between addresses includes each address. invert flag: For a bargraph 1 = Invert, 0 = Non Invert Example: <code>SEND_COMMAND Panel, ''^GIV-500,1''</code> Invert the bargraph.
^GLH	Change the bargraph upper limit. Syntax: <code>''^GLH-<vt addr range>,<bargraph hi>''</code> Variable: - variable text address range = 1 - 4000. - bargraph limit range = 1 - 65535 (bargraph upper limit range). Example: <code>SEND_COMMAND Panel, ''^GLH-500,1000''</code> Changes the bargraph upper limit to 1000.
^GLL	Change the bargraph lower limit. Syntax: <code>''^GLL-<vt addr range>,<bargraph low>''</code> Variable: - variable text address range = 1 - 4000. - bargraph limit range = 1 - 65535 (bargraph lower limit range). Example: <code>SEND_COMMAND Panel, ''^GLL-500,150''</code> Changes the bargraph lower limit to 150.
^GRD	Change the bargraph ramp-down time in 1/10th of a second. Syntax: <code>''^GRD-<vt addr range>,<bargraph ramp down time>''</code> Variable: - variable text address range = 1 - 4000. - bargraph ramp down time = In 1/10th of a second intervals. Example: <code>SEND_COMMAND Panel, ''^GRD-500,200''</code> Changes the bargraph ramp down time to 20 seconds.
^GRU	Change the bargraph ramp-up time in 1/10th of a second. Syntax: <code>''^GRU-<vt addr range>,<bargraph ramp up time>''</code> Variable: - variable text address range = 1 - 4000. - bargraph ramp up time = In 1/10th of a second intervals. Example: <code>SEND_COMMAND Panel, ''^GRU-500,100''</code> Changes the bargraph ramp up time to 10 seconds.

Button Commands																															
^GSC	Change the bargraph slider color or joystick cursor color. A user can also assign the color by Name and R,G,B value (RRGGBB or RRGGBBAA). Syntax: ''^GSC-<vt addr range>,<color value>'' Variable: - variable text address range = 1 - 4000. - color value = Refer to the RGB Triplets and Names For Basic 88 Colors table on page 37. Example: SEND_COMMAND Panel, ''^GSC-500,12'' Changes the bargraph or joystick slider color to Yellow.																														
^GSN	Change the bargraph slider name or joystick cursor name. Slider names and cursor names can be found in the TPDesign4 slider name and cursor drop-down list. Syntax: ''^GSN-<vt addr range>,<bargraph slider name>'' Variable: - variable text address range = 1 - 4000. - bargraph slider name = See table below. <table border="1"><thead><tr><th colspan="3">Bargraph Slider Names:</th></tr></thead><tbody><tr><td>None</td><td>Ball</td><td>Circle -L</td></tr><tr><td>Circle -M</td><td>Circle -S</td><td>Precision</td></tr><tr><td>Rectangle -L</td><td>Rectangle -M</td><td>Rectangle -S</td></tr><tr><td>Windows</td><td>Windows Active</td><td></td></tr></tbody></table> <table border="1"><thead><tr><th colspan="3">Joystick Cursor Names</th></tr></thead><tbody><tr><td>None</td><td>Arrow</td><td>Ball</td></tr><tr><td>Circle</td><td>Crosshairs</td><td>Gunsight</td></tr><tr><td>Hand</td><td>Metal</td><td>Spiral</td></tr><tr><td>Target</td><td>View Finder</td><td></td></tr></tbody></table> Example: SEND_COMMAND Panel, ''^GSN-500,Ball'' Changes the bargraph slider name or the Joystick cursor name to 'Ball'.	Bargraph Slider Names:			None	Ball	Circle -L	Circle -M	Circle -S	Precision	Rectangle -L	Rectangle -M	Rectangle -S	Windows	Windows Active		Joystick Cursor Names			None	Arrow	Ball	Circle	Crosshairs	Gunsight	Hand	Metal	Spiral	Target	View Finder	
Bargraph Slider Names:																															
None	Ball	Circle -L																													
Circle -M	Circle -S	Precision																													
Rectangle -L	Rectangle -M	Rectangle -S																													
Windows	Windows Active																														
Joystick Cursor Names																															
None	Arrow	Ball																													
Circle	Crosshairs	Gunsight																													
Hand	Metal	Spiral																													
Target	View Finder																														
^ICO	Set the icon to a button. Syntax: ''^ICO-<vt addr range>,<button states range>,<icon index>'' Variable: - variable text address range = 1 - 4000. - button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). - icon index range = 0 - 9900 (a value of 0 is clear). Example: SEND_COMMAND Panel, ''^ICO-500.504&510.515,1&2,1'' Sets the icon for On and Off states for buttons with variable text ranges of 500-504 & 510-515.																														

Button Commands										
?ICO	Get the current icon index. Syntax: "'?ICO-<vt addr range>,<button states range>'" Variable: • variable text address range = 1 - 4000. • button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). • custom event type 1003: • Flag - Zero • Value1 - Button state number • Value2 - Icon Index • Value3 - Zero • Text - Blank • Text length - Zero Example: SEND_COMMAND Panel, "'?ICO-529,1&2'" Gets the button 'OFF state' icon index information. The result sent to the Master would be: <pre>ButtonGet Id = 529 Type = 1003 Flag = 0 VALUE1 = 2 VALUE2 = 12 VALUE3 = 0 TEXT = TEXT LENGTH = 0</pre>									
^JSB	Set bitmap/picture alignment using a numeric keypad layout for those buttons with a defined address range. The alignment of 0 is followed by ',<left>,<top>'. The left and top coordinates are relative to the upper left corner of the button. Syntax: "'^JSB-<vt addr range>,<button states range>,<new text alignment>'" Variable: • variable text address range = 1 - 4000. • button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). • new text alignment = Value of 1- 9 corresponds to the following locations: 0 (zero can be used for an absolute position) <table border="1"><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>6</td></tr><tr><td>7</td><td>8</td><td>9</td></tr></table> Example: SEND_COMMAND Panel, "'^JSB-500.504&510.515,1&2,1'" Sets the off/on state picture alignment to upper left corner for those buttons with variable text ranges of 500-504 & 510-515.	1	2	3	4	5	6	7	8	9
1	2	3								
4	5	6								
7	8	9								

Button Commands										
?JSB	Get the current bitmap justification. Syntax: "'?JSB-<vt addr range>,<button states range>'" Variable: • variable text address range = 1 - 4000. • button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). • custom event type 1005: • Flag - Zero • Value1 - Button state number • Value2 - 1 - 9 justify • Value3 - Zero • Text - Blank • Text length - Zero Example: SEND_COMMAND Panel, "'?JSB-529,1'" Gets the button 'OFF state' bitmap justification information. The result sent to the Master would be: <pre>ButtonGet Id = 529 Type = 1005 Flag = 0 VALUE1 = 1 VALUE2 = 5 VALUE3 = 0 TEXT = TEXT LENGTH = 0</pre>									
^JSI	Set icon alignment using a numeric keypad layout for those buttons with a defined address range. The alignment of 0 is followed by '<left>,<top>'. The left and top coordinates are relative to the upper left corner of the button. Syntax: "'^JSI-<vt addr range>,<button states range>,<new icon alignment>'" Variable: • variable text address range = 1 - 4000. • button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). • new icon alignment = Value of 1 - 9 corresponds to the following locations: 0 (zero can be used for an absolute position) <table><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>6</td></tr><tr><td>7</td><td>8</td><td>9</td></tr></table> Example: SEND_COMMAND Panel, "'^JSI-500.504&510.515,1&2,1'" Sets the Off/On state icon alignment to upper left corner for those buttons with variable text range of 500-504 & 510-515.	1	2	3	4	5	6	7	8	9
1	2	3								
4	5	6								
7	8	9								

Button Commands										
?JSI	Get the current icon justification. Syntax: "'?JSI-<vt addr range>,<button states range>'" Variable: • variable text address range = 1 - 4000. • button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). • custom event type 1006: • Flag - Zero • Value1 - Button state number • Value2 - 1 - 9 justify • Value3 - Zero • Text - Blank • Text length - Zero Example: SEND_COMMAND Panel, "'?JSI-529,1'" Gets the button 'OFF state' icon justification information. The result sent to the Master would be: <pre>ButtonGet Id = 529 Type = 1006 Flag = 0 VALUE1 = 1 VALUE2 = 6 VALUE3 = 0 TEXT = TEXT LENGTH = 0</pre>									
^JST	Set text alignment using a numeric keypad layout for those buttons with a defined address range. The alignment of 0 is followed by '<left>,<top>'. The left and top coordinates are relative to the upper left corner of the button. Syntax: "'^JST-<vt addr range>,<button states range>,<new text alignment>'" Variable: • variable text address range = 1 - 4000. • button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). • new text alignment = Value of 1 - 9 corresponds to the following locations: 0 (zero can be used for an absolute position) <table><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>6</td></tr><tr><td>7</td><td>8</td><td>9</td></tr></table> Example: SEND_COMMAND Panel, "'^JST-500.504&510.515,1&2,1'" Sets the text alignment to the upper left corner for those buttons with variable text ranges of 500-504 & 510-515.	1	2	3	4	5	6	7	8	9
1	2	3								
4	5	6								
7	8	9								

Button Commands	
?JST	Get the current text justification. Syntax: <code>''?JST-<vt addr range>,<button states range>''</code> Variable: • variable text address range = 1 - 4000. • button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). • custom event type 1004: • Flag - Zero • Value1 - Button state number • Value2 - 1 - 9 justify • Value3 - Zero • Text - Blank • Text length - Zero Example: <code>SEND_COMMAND Panel, ''?JST-529,1''</code> Gets the button 'OFF state' text justification information. The result sent to the Master would be: ButtonGet Id = 529 Type = 1004 Flag = 0 VALUE1 = 1 VALUE2 = 1 VALUE3 = 0 TEXT = TEXT LENGTH = 0
^MSP	Set the speed of a marquee line. If there was no marquee text enabled for the line, the command is ignored. This works only on general and multistate buttons. Syntax: <code>''^MSP-<vt addr range>,<button states range>,<speed>''</code> Variable: • variable text address range = 1 - 4000. • button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). • Speed of range 1 to 10. 1 = default, slow; 10 = fast. Example: <code>SEND_COMMAND Panel, ''^MSP-529,1,5''</code> Sets the speed for marquee line 529 to 5.
^SCE	Subpage custom event command - Configure subpage custom events. This command can be used to enable or disable the transmission of custom events to the controller whenever certain operations occur. For example, the system programmer may want to be notified whenever a subpage enters the anchor position. The notification mechanism is a custom event. The ^SCE command takes the form of a addr range specifying one or more subpage viewer buttons followed by a comma separated list of custom event numbers. If the number is 0 or blank for a given event type then no custom event will be transmitted when that event occurs. If a number is specified, then it is used as the EVENTID value for the custom event. The range of 32001 to 65535 has been reserved in the panel for user custom event numbers. A different value could be used but might collide with other AMX event numbers. Event configuration is not permanent and all event numbers revert to the default of 0 when the panel restarts.

Button Commands

Syntax:

“'^SCE-<addr range>,<optional anchor event num>,<optional onscreen event num>,<optional offscreen event num>,<optional reorder event num>'”

Variables:

- *address range*: Address codes of buttons to affect. A ‘.’ between addresses includes the range, and & between addresses includes each address. anchor event number: 0 for no event or a value from 32001 to 65535.
- *onscreen event number*: 0 for no event or a value from 32001 to 65535. offscreen event number: 0 for no event or a value from 32001 to 65535.
- *reorder event number*: 0 for no event or a value from 32001 to 65535.

The events are:

- *anchor* - a new subpage has docked in the anchor position.
- *onscreen* - a docking operation has been completed and the subpages in the list are now onscreen. This list will include the anchor along with any subpages that may be partially onscreen.
- *offscreen* - a docking operation has been completed and the subpages in the list are now offscreen.
- *reorder* - the user has reordered the subpages in the set and the list contains all subpages in the new order without regard to onscreen or offscreen state.

In response to any or all of the above events, the panel will create a string which is a list of subpage names separated by a pipe (|) character. The string for the anchor event is a single subpage name. If this string is too long to be transmitted in a single custom event, then multiple custom events will be created and transmitted. If defined, the events are sent in this order when a docking operation completes on a given viewer button: anchor, onscreen, offscreen. If reorder is defined and occurs, it is sent first: reorder, anchor, onscreen, offscreen.

The format of the custom event transmitted to the controller is as follows:

Custom Event Property	Value
Port	port command was received on
ID	address of the button generating the event
Type	the non-zero event number in the ^SCE command
Flag	0
Value 1	which one of possible multiple events this is (1 based)
Value 2	total number of events needed to send the entire string
Value 3	the total size of the original string in bytes
Text	pipe character separated list of subpage names

Example:

SEND_COMMAND Panel, “'^SCE-200,32001,0,0,0'”

If the subpage named TV_Favorite_SyFy enters the anchor position on a subpage viewer button with an address of 200, the following event would be transmitted to the controller when the user had sent this command to the panel:

Custom Event Property	Value
-----------------------	-------

Button Commands		
	Port	port command was received on
	ID	200
	Type	32001
	Flag	0
	Value 1	1
	Value 2	1
	Value 3	16
	Text	TV_Favorite_SyFy
^SHA	Subpage Hide All Command - Hide all subpages in a subpage viewer button. Syntax: <code>''^SHA-<addr range>''</code> Variables: address range: Address codes of buttons to affect. A '.' between addresses includes the range, and & between addresses includes each address. Example: <code>SEND_COMMAND Panel, ''^SHA-200''</code> Remove all subpages from subpage viewer button with address 200.	
^SHD	Subpage Hide Command - This command will hide the named subpage and relocate the surrounding subpages as necessary to close the gap. If the subpage to be hidden is currently offscreen then it is removed without any other motion on the subpage viewer button. Syntax: <code>''^SHD-<addr range>,<name>,<optional time>''</code> Variables: <i>address range:</i> Address codes of buttons to affect. A '.' between addresses includes the range, and & between addresses includes each address. name: name of subpage to hide. If name is __all, then all subpages are hidden. <i>time:</i> Can range from 0 to 30 and represents tenths of a second. This is the amount of time used to move the subpages around when subpages are hidden from a button. Example: <code>SEND_COMMAND Panel, ''^SHD-200,menu1,10''</code> Remove the menu1 subpage from subpage viewer button with address 200 over one second.	

Button Commands	
^SH0	Show or hide a button with a set variable text range. Syntax: <code>''^SH0-<vt addr range>,<command value>''</code> Variable: - variable text address range = 1 - 4000. - command value = (0= hide, 1= show). Example: <code>SEND_COMMAND Panel, ''^SH0-500.504&510.515,0''</code> Hides buttons with variable text address range 500-504 & 510-515.
^SPD	Subpage Padding Command - Set the padding between subpages on a subpage viewer button. Syntax: <code>''^SPD-<addr range>,<padding>''</code> Variables: <i>address range</i>: Address codes of buttons to affect. A '.' between addresses includes the range, and & between addresses includes each address. <i>padding</i>: percentage from 0 to 100 of the first subpage in a set to set as a padding between subpages. For a horizontal subpage viewer button it is a percentage of the width and for a vertical subpage viewer button it is a percentage of the height. Example: <code>SEND_COMMAND Panel, ''^SPD-400,10''</code> Set the padding between subpages in the set to 10% of the dimension of the first subpage in the set.

Button Commands	
^SSH	Subpage Show Command - This command will perform one of three different operations based on the following conditions: - If the named subpage is hidden in the set associated with the viewer button it will be shown in the anchor position. - If the named subpage is not present in the set it will be added to the set and shown in the anchor position. - If the named subpage is already present in the set and is not hidden, then the viewer button will move it to the anchor position. The anchor position is the location on the subpage viewer button specified by its weighting. This will either be left, center or right for horizontal subpage viewer buttons or top, center or bottom for vertical subpage viewer buttons. Surrounding subpages are relocated on the viewer button as needed to accommodate the described operations. Syntax: <code>""^SSH-<addr range>,<name>,<optional position>,<optional time>""</code> Variables: <i>address range</i>: Address codes of buttons to affect. A '.' between addresses includes the range, and & between addresses includes each address. <i>name</i>: Specifies the name of the subpage to be shown or added. <i>position</i>: Specifies where to add (or show) the named subpage in the set with 0 representing the beginning of the set. If this value is left out (or set to 65535) then the weighting value for the viewer button is used to place the new subpage, i.e. left/top, center or right/bottom. When using the weighting locations, set insertion positions can vary based on the current onscreen locations of existing subpages. <i>time</i>: Can range from 0 to 30 and represents tenths of a second. This is the amount of time used to move the subpages around when subpages are added or removed from a button. Example: <code>SEND_COMMAND Panel, ""^SSH-400,media1,0,10""</code> Add or show the media1 subpage in the anchor position over one second.

Button Commands	
^STG	Subpage Toggle Command - If the named subpage is hidden, then this command activates a subpage show command. If the named subpage is present, then a subpage hide command is activated. Syntax: <code>''^STG-<addr range>,<name>,[optional position],[optional time]''</code> Variables: <i>address range:</i> Address codes of buttons to affect. A '.' between addresses includes the range, and & between addresses includes each address. <i>name:</i> Specifies the name of the subpage to be shown or added. <i>position:</i> Specifies where to show the named subpage in the set with 0 representing the beginning of the set. If this value is left out (or set to 65535) then the weighting value for the viewer button is used to place the new subpage, i.e. left/ top, center or right/bottom. When using the weighting locations, set insertion positions can vary based on the current onscreen locations of existing subpages. If the subpage is being hidden this parameter is ignored. <i>time:</i> Can range from 0 to 30 and represents tenths of a second. This is the amount of time used to move the subpages around when subpages are added or removed from a button. Example: <code>SEND_COMMAND Panel, ''^STG-400,media1,0,10''</code> Show or hide the media1 subpage over one second.
^TEC	Set the text effect color for the specified addresses/states to the specified color. The Text Effect is specified by name and can be found in TPD4. You can also assign the color by name or RGB value (RRGGBB or RRGGBBAA). Syntax: <code>''^TEC-<vt addr range>,<button states range>,<color value>''</code> Variable: - variable text address range = 1 - 4000. - button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). - color value = Refer to the RGB Triplets and Names For Basic 88 Colors table on page 37. Example: <code>SEND_COMMAND Panel, ''^TEC-500.504&510.515,1&2,12''</code> Sets the text effect color to Very Light Yellow on buttons with variable text 500-504 and 510-515.

Button Commands	
?TEC	Get the current text effect color. Syntax: <code>''?TEC-<vt addr range>,<button states range>''</code> Variable: • variable text address range = 1 - 4000. • button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). • custom event type 1009: • Flag - Zero • Value1 - Button state number • Value2 - Actual length of string (should be 9) • Value3 - Zero • Text - Hex encoded color value (ex: #000000FF) • Text length - Color name length Example: <code>SEND_COMMAND Panel, ''?TEC-529,1''</code> Gets the button 'OFF state' text effect color information. The result sent to the Master would be: ButtonGet Id = 529 Type = 1009 Flag = 0 VALUE1 = 1 VALUE2 = 9 VALUE3 = 0 TEXT = #5088F2AE TEXT LENGTH = 9
^TEF	Set the text effect. The Text Effect is specified by name and can be found in TPD4. Syntax: <code>''^TEF-<vt addr range>,<button states range>,<text effect name>''</code> Variable: • variable text address range = 1 - 4000. • button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). • text effect name = Refer to the Text Effects table on page Fehler: Verweis nicht gefunden for a listing of text effect names. Example: <code>SEND_COMMAND Panel, ''^TEF-500.504&510.515,1&2,Soft Drop Shadow 3''</code> Sets the text effect to Soft Drop Shadow 3 for the button with variable text range 500-504 and 510-515.

Button Commands	
?TEF	Get the current text effect name. Syntax: <code>''?TEF-<vt addr range>,<button states range>''</code> Variable: • variable text address range = 1 - 4000. • button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). • custom event type 1008: ◦ Flag - Zero ◦ Value1 - Button state number ◦ Value2 - Actual length of string ◦ Value3 - Zero ◦ Text - String that represents the text effect name ◦ Text length - Text effect name length Example: <code>SEND_COMMAND Panel, ''?TEF-529,1''</code> Gets the button 'OFF state' text effect name information. The result sent to the Master would be: ButtonGet Id = 529 Type = 1008 Flag = 0 VALUE1 = 1 VALUE2 = 18 VALUE3 = 0 TEXT = Hard Drop Shadow 3 TEXT LENGTH = 18
^TXT	Assign a text string to those buttons with a defined address range. Sets Non-Unicode text. If the button is an input line, the off state is always the content of the input line and the on state is the placeholder text. Syntax: <code>''^TXT-<vt addr range>,<button states range>,<new text>''</code> Variable: • variable text address range = 1 - 4000. • button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). For input lines: 1 = text of line and 2 = placeholder text. • new text = 1 - 50 ASCII characters. Example 1: <code>SEND_COMMAND Panel, ''^TXT-500.504&510.515,1&2,Test Only''</code> Sets the On and Off state text for buttons with the variable text ranges of 500-504 & 510-515. Example 2: <code>SEND_COMMAND Panel, ''^TXT-500,2,Enter text''</code> Set the On state text for button 500. If this is an input line, it sets the placeholder text to "Enter text". At the moment a character is typed, this text disappears and the typed character is shown.

Button Commands	
?TXT	Get the current text information. If the button is an input line, button state 1 is the content of the input line and button state 2 is a placeholder text. Syntax: "'?TXT-<vt addr range>,<button states range>,<optional index>'" Variable: - variable text address range = 1 - 4000. - button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). - optional index = This is used if a string was too long to get back in one command. The reply will start at this index. - custom event type 1001: - Flag - Zero - Value1 - Button state number - Value2 - Actual length of string - Value3 - Index - Text - Text from the button - Text length - Button text length Example: SEND_COMMAND Panel, "'?TXT-529,1'" Gets the button 'OFF state' text information. The result sent to the Master would be: <pre> ButtonGet Id = 529 Type = 1001 Flag = 0 VALUE1 = 1 VALUE2 = 14 VALUE3 = 1 TEXT = This is a test TEXT LENGTH = 14 </pre>
^UNI	Set Unicode text in the legacy G4 format. For the ^UNI command, the Unicode text is sent as ASCII-HEX nibbles. Note: <i>In the legacy format, Unicode text is always represented in a HEX value. Refer to the TPDesign Instruction Manual for more information.</i> Syntax: "'^UNI-<addr range>,<button states range>,<unicode text>'" Variables: - address range: Address codes of buttons to affect. A '.' between addresses includes the range, and & between addresses includes each address. - button states range: 1 - 256 for multi-state buttons (0 = All states, for General buttons, 1 = Off state and 2 = On state). For input lines: 1 = text of line and 2 = placeholder text. - unicode text: Unicode HEX value. Example: SEND_COMMAND Panel, "'^UNI-500,1,0041'" Sets the button's unicode character to 'A'. SEND_COMMAND TP, "'^UNI-1,0,0041'" Send the variable text 'A' in unicode to all states of the variable text button 1, (for which the character code is 0041 Hex).

Button Commands	
^UTF	Set button state text using UTF-8 text command - Set State Text Command using UTF-8. Assign a text string encoded with UTF-8 (which is ASCII-compatible) to those buttons with a defined address range. While UTF-8 is ASCII compatible, extended ASCII characters in the range 128-255 will be encoded differently based on UTF-8. This command also supports Unicode characters using UTF-8 (which is the encoding method used in >80% of web servers), making the old AMX Hex quad Unicode encoding obsolete. Syntax: <code>''^UTF-<vt addr range>,<button states range>,<new text>''</code> Variables: • variable text address range = 1 - 4000. • Button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). For input lines: 1 = text of line and 2 = placeholder text. • unicode text: Unicode UTF-8 text. Example: <code>SEND_COMMAND Panel, ''^UTF-500.504&510.515,1&2, ASCII ExtendedASCIIÇüéääååç Unicode 動き始めました ''</code> Sets the On and Off state text for buttons with the variable text ranges of 500-504 & 510-515.
^VTP	Simulates a touch/release/pulse at the given coordinate. If the push event is less then 0 or grater than 2 the command is ignored. It is also ignored if the x and y coordinate is out of range. The range must be between 0 and the maximum with and height. Syntax: <code>''^VTP-<push>,<x>,<y>''</code> Variables: • push = Push type where 0 = push, 1 = release and 2 = pulse • x = the x coordinate • y = the y coordinate Example: <code>SEND_COMMAND Panel, ''^VTP-2,32,64''</code> Sends a pulse at coordinate 32, 64.

MVP Panel Lock Passcode Commands

These commands are used to maintain a passcode list. It is possible to assign a user name to a button. If this button has also defined an action (e.g. a page flip), the user is asked for a password before the action is executed. The user entry is just for identifying the passcodes.

The User Password list is an internal list containing user name / passwords values. It is independent from the passwords in the setup. While the passwords in the setup page can be assigned with TPDesign, the User Passwords can not. The only way to assign such a password to a button is the command ^LPB.

MVP Panel Lock Passcode Commands	
^LPB	Assigns a user name to a button. If the button has defined an action (e.g. a page flip), the user is asked for a password before the action is executed. Syntax: "'^LPB-<vt addr range>,<user>'" Variable: vt addr range = variable text address range = 1 – 4000. user = A user name existing in the User Access Password list. Example: SEND_COMMAND Panel, "'^LPB-365,Manager'" Assigns the user <i>Manager</i> to the button 365.
^LPC	Clear all users from the User Access Passwords list. Syntax: "'^LPC'" Example: SEND_COMMAND Panel, "'^LPC'" Clear all users from the User Access Password list on the Password Setup page.
^LPR	Remove a given user from the User Access Passwords list. Syntax: "'^LPR-<user>'" Variable: • user = 1 - 50 ASCII characters. Example: SEND_COMMAND Panel, "'^LPR-Robert'" Remove user named 'Robert' from the User Access Password list.

MVP Panel Lock Passcode Commands	
^LPS	Set the user name and password. This command allows you to: 1. Add a new user name and password OR 2. Set the password for a given user. The user name and password combo is added to the User Access and/or Password list. The user name must be alphanumeric. Syntax: <code>''^LPS-<user>,<passcode>''</code> Variable: • user = 1 - 50 ASCII characters. • passcode = 1 - 50 ASCII characters. Example: <code>SEND_COMMAND Panel, ''^LPS-Manager,undock''</code> Sets a new user name as "Manager" and the password to "undock". Example 2: <code>SEND_COMMAND Panel, ''^LPS-Manager,test''</code> Changes the given user name <i>Manager</i> to "test".

Text Effect Names

The following is a listing of text effects names associated with the ^TEF command.

Text Effects		
Glow -S	Medium Drop Shadow 1	Hard Drop Shadow 1
Glow -M	Medium Drop Shadow 2	Hard Drop Shadow 2
Glow -L	Medium Drop Shadow 3	Hard Drop Shadow 3
Glow -X	Medium Drop Shadow 4	Hard Drop Shadow 4
Outline -S	Medium Drop Shadow 5	Hard Drop Shadow 5
Outline -M	Medium Drop Shadow 6	Hard Drop Shadow 6
Outline -L	Medium Drop Shadow 7	Hard Drop Shadow 7
Outline -X	Medium Drop Shadow 8	Hard Drop Shadow 8
Soft Drop Shadow 1	Medium Drop Shadow 1 with outline	Hard Drop Shadow 1 with outline
Soft Drop Shadow 2	Medium Drop Shadow 2 with outline	Hard Drop Shadow 2 with outline
Soft Drop Shadow 3	Medium Drop Shadow 3 with outline	Hard Drop Shadow 3 with outline
Soft Drop Shadow 4	Medium Drop Shadow 4 with outline	Hard Drop Shadow 4 with outline
Soft Drop Shadow 5	Medium Drop Shadow 5 with outline	Hard Drop Shadow 5 with outline
Soft Drop Shadow 6	Medium Drop Shadow 6 with outline	Hard Drop Shadow 6 with outline
Soft Drop Shadow 7	Medium Drop Shadow 7 with outline	Hard Drop Shadow 7 with outline
Soft Drop Shadow 8	Medium Drop Shadow 8 with outline	Hard Drop Shadow 8 with outline
Soft Drop Shadow 1 with outline		
Soft Drop Shadow 2 with outline		
Soft Drop Shadow 3 with outline		
Soft Drop Shadow 4 with outline		
Soft Drop Shadow 5 with outline		
Soft Drop Shadow 6 with outline		
Soft Drop Shadow 7 with outline		
Soft Drop Shadow 8 with outline		

Panel Runtime Operations

Serial Commands are used in Terminal Emulator mode. These commands are case insensitive.

Panel Runtime Operation Commands	
@AKB ^AKB	Pop up the keyboard icon and initialize the text string to that specified. Keyboard string is set to null on start up and is stored until the program ends. The Prompt Text is optional. Syntax: <code>"@AKB-<initial text>;<prompt text>"</code> Variables: - initial text = 1 - 50 ASCII characters. - prompt text = 1 - 50 ASCII characters. Example: <code>SEND_COMMAND Panel, "'@AKB-Texas;Enter State'"</code> Pops up the Keyboard and initializes the text string 'Texas' with prompt text 'Enter State'.
AKEYB	Pop up the keyboard icon and initialize the text string to that specified. Keyboard string is set to null on start up and is stored until the program ends. Syntax: <code>"AKEYB-<initial text>"</code> Variables: initial text = 1 - 50 ASCII characters. Example: <code>SEND_COMMAND Panel, "'AKEYB-This is a Test'"</code> Pops up the Keyboard and initializes the text string 'This is a Test'.
AKEYP	Pop up the keypad icon and initialize the text string to that specified. The keypad string is set to null on start up and is stored until the program ends. Syntax: <code>"AKEYP-<number string>"</code> Variables: number string = 0 - 9999. Example: <code>SEND_COMMAND Panel, "'AKEP-12345'"</code> Pops up the Keypad and initializes the text string '12345'.
AKEYR	Remove the Keyboard/Keypad. Remove keyboard or keypad that was displayed using 'AKEYB', 'AKEYP', 'PKEYP', @AKB, @AKP, @PKP, @EKP, or @TKP commands. Syntax: <code>"AKEYR"</code> Example: <code>SEND_COMMAND Panel, "'AKEYR'"</code> Removes the Keyboard/Keypad.

Panel Runtime Operation Commands	
@AKP ^AKP	Pop up the keypad icon and initialize the text string to that specified. Keypad string is set to null on start up and is stored until the program ends. The Prompt Text is optional. Syntax: <code>"@AKP-<initial text>;<prompt text>"</code> Variables: - initial text = 1 - 50 ASCII characters. - prompt text = 1 - 50 ASCII characters. Example: <code>SEND_COMMAND Panel, "@AKP-12345678;ENTER PASSWORD"</code> Pops up the Keypad and initializes the text string '12345678' with prompt text 'ENTER PASSWORD'.
@AKR ^AKR	Remove keyboard or keypad that was displayed using 'AKEYB', 'AKEYP', 'PKEYP', @AKB, @AKP, @PKP, @EKP, or @TKP commands. Syntax: <code>"@AKR"</code> Example: <code>SEND_COMMAND Panel, "@AKR"</code> Removes the Keyboard/Keypad.
ABEEP ^ABP	Output a single beep even if beep is Off. Syntax: <code>"ABEEP"</code> or <code>"^ABP"</code> Example: <code>SEND_COMMAND Panel, "ABEEP"</code> Outputs a beep of duration 1 beep even if beep is Off.
ADBEEP	Output a double beep even if beep is Off. Syntax: <code>"ADBEEP"</code> Example: <code>SEND_COMMAND Panel, "ADBEEP"</code> Outputs a double beep even if beep is Off.
BEEP	Output a beep. Syntax: <code>"BEEP"</code> Example: <code>SEND_COMMAND Panel, "BEEP"</code> Outputs a beep.
DBEEP ^ADB	Output a double beep. Syntax: <code>"DBEEP"</code> Example: <code>SEND_COMMAND Panel, "DBEEP"</code> Outputs a double beep.

Panel Runtime Operation Commands	
@EKP ^EKP	Extend the Keypad - Pops up the keypad icon and initializes the text string to that specified. The Prompt Text is optional. Syntax: <code>"@EKP-<initial text>;<prompt text>"</code> Variables: - initial text = 1 - 50 ASCII characters. - prompt text = 1 - 50 ASCII characters. Example: <code>SEND_COMMAND Panel, "@EKP-33333333;Enter Password"</code> Pops up the Keypad and initializes the text string '33333333' with prompt text 'Enter Password'.
^ENC	Set Text Encoding Method - Sets the text encoding method which is used for commands and strings sent from panel to controller (the default is UTF-8). Syntax: <code>"^ENC-<Encoding>"</code> Variables: Encoding: 0: UTF-8 (default), 1: Latin-1 (ISO 8859-1) Example: <code>SEND_COMMAND Panel, "^ENC-1"</code> Sets the encoding method used for all strings to the Controller to Latin-1. Note: NetLinx Studio does not support UTF-8 at this time; therefore UTF-8-encoded characters cannot be copied from TPD5 and pasted in Studio. To use NetLinx Studio to send UTF-8 encoded text, byte values must be enumerated in the command. For example, the following command sends a UTF-8 string to the panel, consisting of ASCII, extended ASCII and Unicode (Chinese) characters: <code>"^UTF-3,0,Hello', \$C3, \$A2, \$C3, \$A3, \$E5, \$9C, \$B0, \$E7, \$9B, \$A4, \$E3, \$83, \$87"</code> Also note that in backwards compatibility mode (i.e. when the ^TXT command is sent or when the ^ENC - 1 command has been sent), ISO-8859-1 is used for character encoding/decoding, since that is what G4 panels used. ISO-8859-1 is different from the Windows-1252 character set in that characters in the range 128-159 (decimal) are non-printable control characters. So in response to a ?TXT query, any characters in that range (assuming the ^ENC - 1 was previously sent) will be returned as AMX Hex quad-encoded values with Custom Event Flag=1, whereas the remainder of the extended ASCII range (160-255) will be returned as Latin-1-encoded characters with Custom Event Flag=0 (see the ISO8859-1 Character Encoding/Decoding table).

Panel Runtime Operation Commands

?ENC

Get Text Encoding Method - Gets the current text encoding method which is used for commands and strings sent from panel to controller (the default is UTF-8).

Syntax:

""?ENC""

Variables: None

Example:

SEND_COMMAND Panel, ""?ENC""

Get the panel's text encoding status. The response returned is a custom event with the following syntax:

Custom Event Property	Value
Port	port command was received on
ID	0
Type	1331
Flag	0
Value 1	encoding (0 = UTF-8, 1 = ISO-8859-1)
Value 2	0
Value 3	0
Text	String that represents the encoding name

Example response for encoding status:

Custom Event Property	Value
Port	port command was received on
ID	1331
Type	0
Flag	0
Value 1	0
Value 2	0
Value 3	0
Text	UTF-8

Panel Runtime Operation Commands																																					
?MAC	Query Panel MAC Address - Query the MAC Address of the panel. Syntax: <code>''?MAC''</code> Variables: None Example: <code>SEND_COMMAND Panel, ''?MAC''</code> Get the panel's MAC Address. The response returned is a custom event with the following syntax: <table border="1"> <thead> <tr> <th>Custom Event Property</th><th>Value</th></tr> </thead> <tbody> <tr> <td>Port</td><td>port command was received on</td></tr> <tr> <td>ID</td><td>0</td></tr> <tr> <td>Type</td><td>1315</td></tr> <tr> <td>Flag</td><td>0</td></tr> <tr> <td>Value 1</td><td>0</td></tr> <tr> <td>Value 2</td><td>0</td></tr> <tr> <td>Value 3</td><td>0</td></tr> <tr> <td>Text</td><td>String that represents the the MAC Address</td></tr> </tbody> </table> Example response: <table border="1"> <thead> <tr> <th>Custom Event Property</th><th>Value</th></tr> </thead> <tbody> <tr> <td>Port</td><td>port command was received on</td></tr> <tr> <td>ID</td><td>0</td></tr> <tr> <td>Type</td><td>1315</td></tr> <tr> <td>Flag</td><td>0</td></tr> <tr> <td>Value 1</td><td>0</td></tr> <tr> <td>Value 2</td><td>0</td></tr> <tr> <td>Value 3</td><td>0</td></tr> <tr> <td>Text</td><td>00:60:9f:90:00:01</td></tr> </tbody> </table>	Custom Event Property	Value	Port	port command was received on	ID	0	Type	1315	Flag	0	Value 1	0	Value 2	0	Value 3	0	Text	String that represents the the MAC Address	Custom Event Property	Value	Port	port command was received on	ID	0	Type	1315	Flag	0	Value 1	0	Value 2	0	Value 3	0	Text	00:60:9f:90:00:01
Custom Event Property	Value																																				
Port	port command was received on																																				
ID	0																																				
Type	1315																																				
Flag	0																																				
Value 1	0																																				
Value 2	0																																				
Value 3	0																																				
Text	String that represents the the MAC Address																																				
Custom Event Property	Value																																				
Port	port command was received on																																				
ID	0																																				
Type	1315																																				
Flag	0																																				
Value 1	0																																				
Value 2	0																																				
Value 3	0																																				
Text	00:60:9f:90:00:01																																				
^MUT	Mute or unmute a panel volume. Syntax: <code>''^MUT-<mute value>''</code> Variables: mute value: 0 for not muted, 1 for muted. Examples: <code>SEND_COMMAND Panel, ''^MUT-1''</code> Mute the master volume. <code>SEND_COMMAND Panel, ''^MUT-0''</code> Unmute the master volume.																																				

Panel Runtime Operation Commands	
PKEYP	Present a private keypad - Pops up the keypad icon and initializes the text string to that specified. Keypad displays a '*' instead of the numbers typed. The Prompt Text is optional. Syntax: <code>"'PKEYP-<initial text>'"</code> Variables: initial text = 1 - 50 ASCII characters. Example: <code>SEND COMMAND Panel, "'PKEYP-123456789'"</code> Pops up the Keypad and initializes the text string '123456789' in '*'.
@PKP	Present a private keypad - Pops up the keypad icon and initializes the text string to that specified. Keypad displays a '*' instead of the numbers typed. The Prompt Text is optional. Syntax: <code>"'@PKP-<initial text>;<prompt text>'"</code> Variables: - initial text = 1 - 50 ASCII characters. - prompt text = 1 - 50 ASCII characters. Example: <code>SEND COMMAND Panel, "'@PKP-1234567;ENTER PASSWORD'"</code> Pops up the Keypad and initializes the text string 'ENTER PASSWORD' in '*'.
^RPP	Reset protected password command - This command is used to reset the protected setup password to the factory default value. Syntax: <code>"'^RPP'"</code> Variables: None Example: <code>SEND_COMMAND Panel, "'^RPP'"</code> Reset the panel protected password to the factory default.
SETUP ^STP	Send panel to SETUP page. Syntax: <code>"'SETUP'"</code> Example: <code>SEND COMMAND Panel, "'SETUP'"</code> Sends the panel to the Setup Page.
SHUTDOWN	Shut down the program. Syntax: <code>"'SHUTDOWN'"</code> Example: <code>SEND COMMAND Panel, "'SHUTDOWN'"</code> Ends the application.

Panel Runtime Operation Commands	
@SOU ^SOU	Play a sound file. Syntax: "@SOU-<sound name>" Variables: sound name = Name of the sound file. Supported sound file formats are: WAV & MP3. Example: SEND COMMAND Panel, "'@SOU-Music.wav'" Plays the 'Music.wav' file.
@TKP ^TKP	Present a telephone keypad - Pops up the keypad icon and initializes the text string to that specified. The Prompt Text is optional. Syntax: "@TKP-<initial text>;<prompt text>" Variables: - initial text = 1 - 50 ASCII characters. - prompt text = 1 - 50 ASCII characters. Example: SEND COMMAND Panel, "'@TKP-999.222.1211;Enter Phone Number'" Pops-up the Keypad and initializes the text string '999.222.1211' with prompt text 'Enter Phone Number'.
@VKB	Popup the virtual keyboard. Syntax: "@VKB" Example: SEND COMMAND Panel, "'@VKB'" Pops-up the virtual keyboard.

Input Commands

These Send Commands are case insensitive.

Input Commands	
^KPS	Set the keyboard passthru. Syntax: <code>''^KPS-<pass data>''</code> Variable: pass data: • <blank/empty> = Disables the keyboard. • 0 = Pass data to G4 application (default). This can be used with VPC or text areas. • 1 - 4 = Not used. • 5 = Sends out data to the Master. Example: <code>SEND_COMMAND Panel, ''^KPS-5''</code> Sets the keyboard passthru to the Master. Option 5 sends keystrokes directly to the Master via the Send Output String mechanism. This process sends a virtual keystroke command (^VKS) to the Master. Example 2: <code>SEND_COMMAND Panel, ''^KPS-0''</code> Disables the keyboard passthru to the Master.
^VKS	Send one or more virtual key strokes to the G4 application. Key presses and key releases are not distinguished except in the case of CTRL, ALT, and SHIFT. Refer to the Embedded Codes table on page 115 that defines special characters which can be included with the string but may not be represented by the ASCII character set. Syntax: <code>''^VKS-<string>''</code> Variable: • string = Only 1 string per command/only one stroke per command. Example: <code>SEND_COMMAND Panel, ''^VKS- '8''</code> Sends out the keystroke 'backspace' to the G4 application.

Dynamic Image Commands

The following table describes Dynamic Image Commands.

Dynamic Image Commands	
^BBR	Set the bitmap of a button to use a particular resource. Syntax: <code>''^BBR-<vt addr range>,<button states range>,<resource name>''</code> Variable: - variable text address range = 1 - 4000. - button states range = 1 - 256 for multi-state buttons (0 = All states, for General buttons 1 = Off state and 2 = On state). - resource name = 1 - 50 ASCII characters. Example: <code>SEND_COMMAND Panel, ''^BBR-700,1,Sports_Image''</code> Sets the resource name of the button to 'Sports_Image'.
^RAF	Add new resources - Adds any and all resource parameters by sending embedded codes and data. Since the embedded codes are preceded by a '%' character, any '%' character contained in the URL must be escaped with a second '%' character (see example). The file name field (indicated by a %F embedded code) may contain special escape sequences as shown in the ^RAF, ^RMF - <i>Embedded Codes</i> table below. Syntax: <code>''^RAF-<resource name>,<data>''</code> Variables: - resource name = 1 - 50 ASCII characters. - data = Refers to the embedded codes, see the ^RAF, ^RMF Example: <code>SEND_COMMAND Panel, ''^RAF-New Image,%P0%HAMX.COM%Alab/Test/file%Ftest.jpg''</code> Adds a new resource. The resource name is 'New Image' %P (protocol) is an HTTP %H (host name) is AMX.COM %A (file path) is Lab/Test_f ile %F (file name) is test.jpg.
^RFR	Force a refresh for a given resource. Syntax: <code>''^RFR-<resource name>''</code> Variable: resource name = 1 - 50 ASCII characters. Example: <code>SEND_COMMAND Panel, ''^RFR-Sports_Image''</code> Forces a refresh on 'Sports_Image'.

Dynamic Image Commands	
^RMF	Modify an existing resource - Modifies any and all resource parameters by sending embedded codes and data. Since the embedded codes are preceded by a '%' character, any '%' character contained in the URL must be escaped with a second '%' character (see example). The file name field (indicated by a %F embedded code) may contain special escape sequences as shown in the ^RAF, ^RMF Syntax: ""^RMF-<resource name>,<data>'" Variables: • resource name = 1 - 50 ASCII characters • data = Refers to the embedded codes, see the ^RAF, ^RMF Example: SEND_COMMAND Panel, ""^RMF-Sports_Image,%ALab/Test/Images%Ftest.jpg'" Changes the resource 'Sports_Image' file name to 'test.jpg' and the path to 'Lab_Test/Images'.
^RSR	Change the refresh rate for a given resource. Syntax: ""^RSR-<resource name>,<refresh rate>'" Variable: • resource name = 1 - 50 ASCII characters. • refresh rate = Measured in seconds. Example: SEND_COMMAND Panel, ""^RSR-Sports_Image,5'" Sets the refresh rate to 5 seconds for the given resource ('Sports_Image').

Subpages commands

Subpages commands	
^EPR	Execute Push on Release. Syntax: ^EPR-<addressArray>,<state> This sets the subpage viewer in a mode where a push will not be sent to the master until a touch release is received. Any movement of the finger during the screen press will cause neither the press nor the release to be sent. State 0 is off, State 1 is on. Example: SEND_COMMAND 10001:1:0,'^EPR-401,1'
^LVD	Set Listview Data Source - This command sets the data source to drive the Listview entries. Note that this command only configures the data source it does not actually cause the data to be fetched. The ^LVR refresh command must be issued to load the data. Syntax: "'^LVD-<addr range>,<URL to data source or Dynamic Data Resource name>,<configuration_option=configuration_value>'" Variable: <i>address range</i>: Address codes of buttons to affect. A '.' between addresses includes the range, and & between addresses includes each address. Data source URL/Dynamic Data Resource name (required): If the suffix of the URL is .csv or .CSV then the URL will be assumed to point to a csv file. Otherwise the type is assumed to be the XPort amxstandard.xml format. Supported URL schemes are HTTP, HTTPS, and FILE. Data Source URL Notes: HTTPS is supported in G5 panel firmware version v1.4.9 and higher. HTTPS is not supported by TPDesign5 dynamic image resources at this time. A file on the panel's local filesystem can be specified using the file:/// option. There must be three forward slashes after 'file:'. An FTP URL scheme is not supported. Refer to Notes on Using Image URLs With Listview Buttons for additional details. option list: a optional comma separated list of one or more configuration parameters followed by an equal sign and the configuration setting. Configuration Options: <i>user</i> - The user name to use for authenticating to the web server when retrieving the feed data source file. If specified when URL is a Dynamic Data Resource, this value will override the username inside the Dynamic Data Resource. Note: <i>For server authentication to occur, the Username (user) and Password (pass) must be included in the ^LVD command, and they must match the credentials required by the server.</i> <i>pass</i> - The password to use for authenticating to the web server when retrieving the feed data source file. If specified when URL is a Dynamic Data Resource, this value will override the password inside the Dynamic Data Resource.

Subpages commands

	Note: For server authentication to occur, the Username (user) and Password (pass) must be included in the ^LVD command, and they must match the credentials required by the server. csv - a boolean indicating whether or not to parse the data source as a CSV file. If not present, defaults to false. • <i>has_headers</i> - a boolean indicating that the first line of the CSV file has column headers which will be used to name the content fields for each data record. If true it automatically implies that csv is also true. If this option is not present then the default for a CSV file is false. In the absence of headers, the content fields will be named using the following convention: column1, column2, column3... (CSV files only, since XML always has field names specified within the file). Example: SEND_COMMAND Panel, ""^LVD-42,http://192.168.220.231/public/lv42data.csv,has_headers=1"" Configures the Listview button to use the CSV file at the URL as its data source. The first line of the CSV file should be parsed as field names and not as Listview entry record data.														
^LVE	Set ListView custom event number - This command sets the custom event number reported by Listview refresh operations. Syntax: ""^LVE-<addr range>,<Listview custom event number>"" Variable: address range: Address codes of buttons to affect. A '.' between addresses includes the range, and & between addresses includes each address. event number: The custom event number to report Listview events. At this time, only refresh events are reported. A value of 0 turns off custom event reporting, A value > 0 assigns the value to the Listview custom event number for that address. The default value is 1401 (custom events reported). When enabled, the custom event format reported is: <table border="1"> <thead> <tr> <th>Custom Event Property</th><th>Value</th></tr> </thead> <tbody> <tr> <td>Port</td><td>port command was received on</td></tr> <tr> <td>ID</td><td>address of the button</td></tr> <tr> <td>Type</td><td>button event number set by ^LVE</td></tr> <tr> <td>Flag</td><td>StartRefresh = 1; FinishRefresh = 2; Error = 0xffff (\$FFFF);</td></tr> <tr> <td>Value 1</td><td> <u>If flag is StartRefresh (1) or FinishRefresh (2):</u> InitRefresh = 0; (refresh by dynamic resource) ManualRefresh = 1; (refresh by send command) TimedRefresh = 2; (refresh by timer) <u>If flag is Error:</u> Error = -1; (some form of error, see custom.text for description) InvalidUrl = -2; (URL is null, should never happen) LoginFailed = -3; (could not authenticate to web server). </td></tr> <tr> <td>Value 2</td><td>data load id. Every data load is assigned a unique id that</td></tr> </tbody> </table>	Custom Event Property	Value	Port	port command was received on	ID	address of the button	Type	button event number set by ^LVE	Flag	StartRefresh = 1; FinishRefresh = 2; Error = 0xffff (\$FFFF);	Value 1	<u>If flag is StartRefresh (1) or FinishRefresh (2):</u> InitRefresh = 0; (refresh by dynamic resource) ManualRefresh = 1; (refresh by send command) TimedRefresh = 2; (refresh by timer) <u>If flag is Error:</u> Error = -1; (some form of error, see custom.text for description) InvalidUrl = -2; (URL is null, should never happen) LoginFailed = -3; (could not authenticate to web server).	Value 2	data load id. Every data load is assigned a unique id that
Custom Event Property	Value														
Port	port command was received on														
ID	address of the button														
Type	button event number set by ^LVE														
Flag	StartRefresh = 1; FinishRefresh = 2; Error = 0xffff (\$FFFF);														
Value 1	<u>If flag is StartRefresh (1) or FinishRefresh (2):</u> InitRefresh = 0; (refresh by dynamic resource) ManualRefresh = 1; (refresh by send command) TimedRefresh = 2; (refresh by timer) <u>If flag is Error:</u> Error = -1; (some form of error, see custom.text for description) InvalidUrl = -2; (URL is null, should never happen) LoginFailed = -3; (could not authenticate to web server).														
Value 2	data load id. Every data load is assigned a unique id that														

Subpages commands		
		counts up from 0. This is used to correlate StartRefresh/FinishRefresh/Error events on particular addresses.
	Value 3	When Custom.flag == FinishRefresh, this is the number of records in list. Otherwise is 0.
	Text	feed URL string, or error message if flag is Error
	Example: SEND_COMMAND Panel, ""^LVE-42,1401"" Configures the Listview widget to send Listview custom events on event 1401.	
^LVF	Listview Filter - This command can be used to programmatically change the filter contents of the Listview widget. When the filter contents is changed, the filter will be applied to the current Listview data which can change the number of items displayed based on those that meet the filter sequence. The filter changes immediately, and the filter can be set or cleared with this command.	
	Syntax: ""^LVF-<addr range>,<filter character sequence>"" Variable: <i>address range:</i> Address codes of buttons to affect. A '.' between addresses includes the range, and & between addresses includes each address. <i>filter character sequence:</i> All characters including white space characters will be applied to the filter. Example: SEND_COMMAND Panel, ""^LVF-42,amx"" Sets the filter sequence to amx. Only items in the data set that contain the sequence amx will be displayed. SEND_COMMAND Panel, ""^LVF-42, "" Clears the filter sequence. All items in the data set can be viewed in the Listview.	
^LVL	Listview Layout - This command sets the layout configuration to configure the visual representation of the Listview entries.	
	Syntax: ""^LVL-<vt addr range>,<layout_option=layout_value>"" Variables: <i>Variable text address range:</i> 1 – 4000. A comma separated list of one or more layout configuration parameters followed by an equal sign and the configuration setting. <i>Layout Options:</i> columns - Number of columns parameter. An integer that represents the number of columns to display. The number must be at least 1 and a value that exceeds the minimum cell width will truncate to the maximum. Note: Valid tags for the columns parameter are columns=, nc=, and numcol=.comp - Component parameter. An integer that is a value which determines which graphical components are present in the cell. When the component values are bitwise or'd together, it creates the encoding for the cell components that are populated. If a configuration parameter is not in the current command, the last value for the configuration parameter is used.	

Subpages commands	
Note: Valid tags for the comp parameter are c= and comp=.	
Component Value	Description
1	The image (i) is used in the cell.
2	The primary text field (t1) is used in the cell.
4	The secondary text field (t2) is used in the cell
<i>Not all variations of component values are valid. To have the secondary text field present, the primary text field must be preset as well.</i>	
Component Description	Combinations
0	Invalid. No component displayed.
1	The image (i) is the only component displayed.
2	The primary text field (t1) is the only component displayed.
3	The image (i) and the primary text field (t1) are displayed.
4	Secondary text (t2) only. Invalid. Secondary text (t2) cannot be displayed without the primary text (t1).
5	Secondary text (t2) and image (i). Invalid. Secondary text (t2) cannot be displayed without the primary text (t1).
6	The primary text (t1) and secondary text (t2) are displayed.
7	The image (i), primary text (t1), and secondary text (t2) are displayed
cellheight - An integer or percentage that sets the height of a cell. The value can be an integer >= the minimum cell height (48), or a percentage of the list height (5% up to 95%). To specify a percentage, append a '%' to the end of the value. Note: Valid tags for the cellheight param are ch= and cellheight=.	
layout - An integer that sets the layout configuration of each cell. Note: valid tags for the layout parameter are l= and layout=.	
Layout Value	Description
1	Horizontal layout with image on the left and text(s) on the right. If multiple texts are selected then the texts are stacked vertically.
2	Horizontal layout with text1 on the left, image in the center, and text2 on the right. If multiple texts are selected then the texts are stacked vertically.
3	Vertical layout with the image on the top and text(s) below the image. If multiple texts are selected then text1 is below the image and text2 is below text1.
4	Vertical layout with the image on the bottom and text(s) above the image. If multiple texts are selected then text1 is on top, text2 is below text1, and the image is below text2.

Subpages commands		
	5	Vertical layout with text1 on top, the image below text1, and text2 below the image.
	6	Vertical layout with text1 on top, the image below text1, and text2 below the image.
	p1 - layout percentage 1. Sets the boundaries between cell components in different layouts. An integer between 10 and 90 that sets the boundary between components as a percentage of the cell dimension. The percentage can be specified as a number between 5-95 with an optional percentage sign '%' at the end. p2 - layout percentage 2. Sets the boundaries between cell components in different layouts. An integer between 10 and 90 that sets the boundary between components as a percentage of the cell dimension. The percentage can be specified as a number between 5-95 with an optional percentage sign '%' at the end. filter - Enable or disable the search filter on the Listview. To enable set to 'true', 'on', or '1'. To disable set to 'false', 'off', or '0'. Note: Valid tags for the filter parameter are <i>f=</i> and <i>filter=</i>. <i>filterheight</i> - An integer or percentage that sets the height of the filter in the Listview. The value can be an integer >= the minimum filter height (24), or a percentage of the list height (5% to 25%). To specify a percentage, append a '%' to the end of the value. Note: Valid tags for the filterheight param is <i>fh=</i> and <i>filterheight=</i>. <i>alphascroll</i> - Enable or disable the alpha scroll on the Listview. To enable set to 'true', 'on', or '1'. To disable set to 'false', 'off', or '0'. Note: Valid tags for the alphascroll parameter are <i>as=</i> and <i>alphascroll=</i>. Examples: <code>SEND_COMMAND Panel,"'^LVL-42, layout=1, comp=7, columns=1, cellheight=120, p1=40%, p2=66%'"</code> Sets the Listview configuration display an image and 2 text fields (comp=7), in a layout 1 configuration (layout=1 horizontal layout of the image on left and text1 and text2 to the right of the image). There is 1 column (columns=1) and the cell is 120 pixels high (h=120). The image width will be 40% of the cell width (p1=40%) with text1 and text2 having a width of 60% of the cell width. The height of text1 will be 66% of the cell height (p2=66%) with text2 height of 34% of the cell height. <code>SEND_COMMAND Panel,"'^LVL-42,l=4, c=3, ch=150, nc=4, p1=70'"</code> Sets the Listview configuration display an image and 1 text fields (c=4), in a layout 4 configuration (l=4 vertical layout of the image on top and text1 below the image). There are 4 columns (nc=4) and the cell is 150 pixels high (ch=150). The image height will be 70% of the cell height (p1=70) with text1 having a height of 30% of the cell height. <code>SEND_COMMAND Panel,"'^LVL-42,layout=3, comp=6, ch=100, numcol=1, p1=50'"</code> Sets the Listview configuration display 2 text fields (comp=6), in a layout 3 configuration (layout=2 horizontal layout of text1 on the left and text2 on the right). There is 1 column (numcol=1) and the cell is 100 pixels high (ch=100). The text1 width will be 50% of the cell width (p1=50) with text2 having a width of 50% of the cell width. <code>SEND_COMMAND Panel,"'^LVL-42,filter=1, fh=10%, as=false'"</code> Sets the Listview search filter enabled (filter=1), the search filter textview height to 10% of the Listview height (fh=10%), and disables the alphascroller on the Listview.	
^LVM	Listview Map Fields - This command maps the fields from the data source to the	

Subpages commands	
	display elements of a Listview entry. Each list entry corresponds to a record if the data came from the NetLinx data access API or XPort. If the data source is a csv file, then each list entry corresponds to a row in the file. A list entry can have up to two lines of text and a URL that points to an image. Each display element for a list entry has to be mapped to a field in the record. If no mapping is specified, then a default mapping is used which is simply to map the fields in order based on the screen layout of the list entry. So, if the list type was an image and two lines of text, the first content field in the record would be interpreted as the URL to the image, the next field would be the first line of text and the next field would be the second line of text. To override this default behavior, the ^LVM command should be used to specify the correct mapping. Syntax: <code>""^LVM-<addr range>,<display_element=field_expression <display_element=field_expression> ...""</code> Variable: • <i>address range</i>: Address codes of buttons to affect. A '.' between addresses includes the range, and & between addresses includes each address. • <i>display element list</i>: A pipe character " " separated list of mapping expressions. A pipe is used because typical field expressions may use more common characters such as the comma or semicolon. Display Elements: t1 - the first text element t2 - the second text element i1 - the first image future display types may support more text and image elements which will follow the same convention: t3... i2... Field Expressions: An expression that can be used to map field values to display elements. Any time a field name is used, it follows the form \${field_name}. Other text characters can be used to construct a more complex string using multiple fields. Examples: <code>SEND_COMMAND Panel,"'^LVM-42,i1=\${image}''"</code> Configures the Listview widget to map an image field to the image display element. In this example, the Listview type is assumed to be a single image only. <code>SEND_COMMAND Panel,"'^LVM-42,i1=\${image} t1=\${lname}, \${fname} t2=\${number}''"</code> The Listview widget is the type that has an image and two lines of text. The top line will consolidate two different fields in the form of last name, first name. The second line of text will be the phone number. <code>SEND_COMMAND Panel,"'^LVM-42,t1=\${column2}, \${column1} t2=\${column3} i1=\${column4}''"</code> This is the same example as the one above it but the source of the data
^LVN	Listview Navigate - This command can be used to move the Listview widget. Navigation commands will be range checked. The command will attempt to position the specified list entry on the top line of the Listview widget. When navigating at the end of the list, however, the widget will position the last item in the list on the bottom line and will not leave blank lines at the bottom. The only exception to this case will be when the Listview has fewer entries than the number of displayable entries. If the optional select boolean is present, and the navigation command used support the select option, the item at the destination will be selected and a item selected custom event

Subpages commands	
	will be initiated. Syntax: <code>""^LVN-<addr range>,<navigation_command>[,<boolean_select_param>]""</code> Variables: <i>address range</i>: Address codes of buttons to affect. A '.' between addresses includes the range, and & between addresses includes each address. navigation command. optional select boolean <i>Navigation Commands</i>: t or T - move to the top of the list (supports an optional select boolean). b or B - move to the bottom of the list (supports an optional select boolean). d or D - page down (DOES NOT support the optional select boolean. A select boolean will be ignored if present). n - move to a specific list entry number at position n. n is a zero based index. (supports an optional select boolean). (Note: If n is < 0 and select is true then the current selected item is deselected.) u or U - page up (DOES NOT support the optional select boolean. A select boolean will be ignored if present). Examples: <code>SEND_COMMAND Panel,"'^LVN-42,B'"</code> Move to the bottom of the list. <code>SEND_COMMAND Panel,"'^LVN-42,d'"</code> Move the list down a page. <code>SEND_COMMAND Panel,"'^LVN-42,3,1'"</code> Move the list to position 3 in the list and select the item at position 3.
^LVR	Listview Refresh Data - This command has two different functions. If it is sent without any parameters, it causes the Listview widget to load data from its configured data source. If optional parameters are included with the command, then the automatic data refresh options are configured. The typical behavior for auto refresh is that the last modified time of the data source is tracked. At the refresh interval, the last modified time of the data source is compared against the stored value. If the data is newer, then it is reloaded and the Listview widget is refreshed with the updated data. If the data is unchanged, then it is not reloaded. The default for auto refresh is off. Syntax: <code>""^LVR-<addr range>[,<refresh_interval>,<force_reload>]""</code> Variable: <i>address range</i>: Address codes of buttons to affect. A '.' between addresses includes the range, and & between addresses includes each address. <i>refresh_interval</i> - the optional interval (in seconds) at which to check for newer data. 0 (the default) means auto refresh is off. Minimum is 5 seconds. If not specified, the current refresh interval is retained. <i>force_reload</i> - the optional parameter to force the Listview to ignore and data file timestamps and to force a clear on image caches for refreshed Listview images. Not specified or 0 will not force a reload, 1 will force a reload of data file and images associated with data file. Note: <i>This can cause the images in a Listview to flicker upon the reload. This is the expected behavior due to the images being reloaded from the server.</i>

Subpages commands	
	Examples: <code>SEND_COMMAND Panel,"'^LVR-42'"</code> Commands the Listview widget to load the data from the data source and populate the Listview display widget. <code>SEND_COMMAND Panel,"'^LVR-42,15'"</code> Commands the Listview widget to check for an updated data source every 15 seconds. <code>SEND_COMMAND Panel,"'^LVR-42,600,1'"</code> Commands the Listview widget to check for an updated data source every hour, and to force a reload of the data and the images.
^LVS	Listview Sort Data - This command sets the columns that are used for sorting of lists, as well as the type of sorting that is done. The multiple columns are allowed in the sort procedure. The order of the columns in the command determine the order of the sorting. The first column is the primary sorting data, the second would be used for sorting with rows of data that are equal in the primary columns, and so on for however many columns are used for sorting. If no columns are listed in the command, then the current sorting columns are used if they have been previously defined. The type of sort is an optional part of the command and follows the sort columns. Initially, there are four different sort types available. None (n) - No sorting is performed. Ascending (a) - Ascending sort using localized character weighting. Descending (d) - Descending sort using localized character weighting. Override (*) - Override sort syntax portion of command determines sorting. The override sort syntax allows for complex SQLite ORDER BY syntax for sorting. When override is selected, the sort columns that were set in the command or previously are ignored and the entire sorting statement must be in the override sort syntax. The words ORDER BY should not be in the syntax. They are inserted by the firmware. Syntax: <code>"'^LVS-<addr range>,<primary sort column name, secondary sort column name,..., final sort column name>[;<sort type>[;<override sort syntax>]]'"</code> Variables: <i>address range</i>: Address codes of buttons to affect. A '.' between addresses includes the range, and & between addresses includes each address. Sort columns - comma separated list of sort columns in the order of sort priority. Sort columns can be specified using the \${column name} syntax that is used in the ^LVM command. Columns can be Content Fields or Metadata Fields in the controller Datafeed XML file generated by the controller. Metadata fields are prepended with "meta" in front of the "label" attribute of the field. <i>Sort Type</i> - A character indicating the sorting algorithm to use. 'a' – ascending 'd' – descending '*' - override. Sort command syntax must follow in the next part of the command. 'n' - none (default). Any character that is not a,d, or * will set sort to none. <i>Override sort syntax</i> - A SQLite ORDER BY statement to use as the sort. Examples:

Subpages commands	
	<code>SEND_COMMAND Panel,"'^LVS-42, \${artist name},\${title};a '"</code> Commands the Listview widget to sort the data source by the artist name and then title in an ascending order. Equates to "artistname, title COLLATE LOCALIZED ASC" override syntax. <code>SEND_COMMAND Panel,"'^LVS-42, \${artist name},\${title};d '"</code> Commands the Listview widget to sort the data source by the artist name and then title in an descending order. Equates to "artistname COLLATE LOCALIZED DESC, title COLLATE LOCALIZED DESC" override syntax. <code>SEND_COMMAND Panel,"'^LVS-42,;n'"</code> Commands the Listview widget to not sort the current data. <code>SEND_COMMAND Panel,"'^LVS-150,\${user name},\${text};*;meta\${Record timestamp} ASC'"</code> Commands the panel to sort by the meta data field Record timestamp in ASCENDING order. The username and test fields are ignored. <code>SEND_COMMAND Panel,"'^LVS-150,;*;meta\${Record timestamp} ASC'"</code> Commands the panel to sort by the meta data field "Record timestamp" in ASCENDING order. The username and test columns are ignored. <code>SEND_COMMAND Panel,"'^LVS-150,;*;LENGTH(\${description}),\${description} ASC'"</code> Command the panel to sort by the number of characters in the description field, and then by the contents of the description field in ASCENDING order.
^SCE	Configures subpage custom events. Syntax: <code>"'^SCE-<vt addr range>,<optional anchor event num>,<optional onscreen event num>,<optional offscreen event num>,<optional reorder event num>'"</code> This command can be used to enable or disable the transmission of custom events to the master whenever certain operations occur. For example, the system programmer may want to be notified whenever a subpage enters the anchor position. The notification mechanism is a custom event. The ^SCE command takes the form of a vt addr range specifying one or more subpage viewer buttons followed by a comma separated list of custom event numbers. If the number is 0 or blank for a given event type then no custom event will be transmitted when that event occurs. If a number is specified, then it is used as the EVENTID value for the custom event. The range of 32001 to 65535 has been reserved in the panel for user custom event numbers. A different value could be used but might collide with other AMX event numbers. Event configuration is not permanent and all event numbers revert to the default of 0 when the panel restarts. Events: • Anchor - a new subpage has docked in the anchor position • Onscreen - a docking operation has been completed and the subpages in the list are now onscreen. This list will include the anchor along with any subpages that may be partially onscreen. • Offscreen - a docking operation has been completed and the subpages in the list are now offscreen. • Reorder - the user has reordered the subpages in the set and the list contains all subpages in the new order without regard to onscreen or offscreen state. In response to any or all of the above events, the panel will create a string which is a list of subpage names separated by a pipe () character. The string for the anchor event is a single subpage name. If this string is too long to be transmitted in a single custom event, then multiple custom

Subpages commands	
	events will be created and transmitted. The format of the custom event transmitted to the master is as follows: CUSTOM.TYPE = EVENTID = the non-zero event number in the ^SCE command CUSTOM.ID = ADDRESS = the address of the viewer button which generated the event CUSTOM.FLAG = 0 CUSTOM.VALUE1 = which one of possible multiple events this is (1 based) CUSTOM.VALUE2 = total number of events needed to send the entire string CUSTOM.VALUE3 = the total size of the original string in bytes CUSTOM.TEXT = pipe character separated list of subpage names As an example, if the subpage named TV_Favorite_SyFy enters the anchor position on a subpage viewer button with an address of 200, the following event would be transmitted to the master when the user had sent this command to the panel: ^SCE-200,32001,0,0,0 CUSTOM.TYPE = EVENTID = 32001 CUSTOM.ID = ADDRESS = 200 CUSTOM.FLAG = 0 CUSTOM.VALUE1 = 1 CUSTOM.VALUE2 = 1 CUSTOM.VALUE3 = 16 CUSTOM.TEXT = TV_Favorite_SyFy If defined, the events are sent in this order when a docking operation completes on a given viewer button: anchor, onscreen, offscreen. If reorder is defined and occurs, it is sent first: reorder, anchor, onscreen, offscreen
^SDR	Enabling subpage dynamic reordering. Syntax: "^SDR-<vt addr range>,<enable state>,<optional hold time>" This command can be used to enable or disable dynamic reordering for a given viewer button or set of viewer buttons. It can also be used to set the amount of time to wait before initiating the single finger reorder time. Variables: • enable state – This value can be either "on" or "ON" or "1" to enable dynamic reordering for the specified viewer button(s). Any other value will disable dynamic reordering for the specified viewer button(s). • hold time – This value is in tenths of a second. The value will be rounded up to the next highest quarter of a second. This is the amount of time that the user must press and hold a subpage with a single finger to trigger a dynamic reordering operation.

SIP Commands

Panel to Master

The following table lists and describes SIP commands that are generated from the touch panel.

SIP Commands – Panel to Master	
^PHN-AUTOANSWER	SIP auto answer status - Provides the state of the auto-answer feature. Syntax: "'^PHN-AUTOANSWER,<state>'" Variable: state = 0 or 1 (off or on) Example: ^PHN-AUTOANSWER,1 The panel sent a command status to the master indicating the auto-answer is on.
^PHN-CALL	SIP call progress status - Provides call progress notification for a call. Syntax: "'^PHN-CALL,<status>,<connection id>'" Variables - status = CONNECTED, DISCONNECTED, TRYING, RINGING, or HOLD. - connection id = The identifying number of the connection. Example: ^PHN-CALL,CONNECTED,1 Notifies that the call is connected.
^PHN-IM	SIP instant message – Provides incoming instant message. Syntax: "'^PHN-IN,<from>,<msg>'" Variables: - from = The SIP URI of the device the message is coming from. - msg = A short message. The message is limited to 256 bytes. Any longer message is cut off. Example: ^PHN-IM,9001@127.0.0.1,A short message The panel received the message “A short message” from 9001@127.0.0.1.
^PHN-INCOMING	SIP incoming call status - Provides incoming call notification and the connection ID used for all future commands related to this call. The connection id will be 0 or 1. Syntax: "'^PHN-INCOMING,<caller number>,<caller name>,<connection id>, <timestamp>'" Variables: - caller number = The phone number of the incoming call - caller name = The name associated with the caller number - connection id = The identifying number of the connection - timestamp = The current time in MM/DD/YY HH:MM:SS format Example: ^PHN-INCOMING,"1235556789",MAIN,1,01/01/2011 11:11:11

SIP Commands – Panel to Master	
	The panel sent a command status to the master indicating an incoming call from number 1235556789 named MAIN at Jan 1, 2011 at 11:11:11.
^PHN-LINESTATE	SIP call linestate status - Indicates the current state of each of the available connections used to manage calls. Syntax: <code>''^PHN-LINESTATE,<connection id>,<state>,<connection id>,<state>,..., SIP,<extn>''</code> Variables: • connection id = The identifying number of the connection. • state = IDLE, HOLD, or CONNECTED • extn = The local extension of this panel (see Example) Example: <code>^PHN-LINESTATE,1,IDLE,2,CONNECTED,SIP,1234</code> The panel sent a command status to the master indicating line 1 is idle and line 2 is connected and this is extension 1234.
^PHN-MSGWAITING	SIP call message waiting status - Indicates the number of messages waiting the user's voice mail box. Syntax: <code>''^PHN-MSGWAITING,<messages>,<new message count>,<old message count>,<new urgent message count>,<old urgent message count>''</code> Variables: • messages = 0 or 1 (1 indicates new messages) • new message count = The number of new messages. • old message count = The number of old messages. • new urgent message count = The number of new messages marked urgent. • old urgent message count = The number of old messages marked urgent. Example: <code>^PHN-MSGWAITING,1,1,2,1,0</code> The panel sent a command status to the master indicating there are calls waiting (1 new, 2 old, 1 new urgent, 0 old urgent).
^PHN-PRIVACY	SIP call privacy status - Indicates the state of the privacy feature. Syntax: <code>''^PHN-PRIVACY,<state>''</code> Variables: • state = 0 (Disable) or 1 (Enable) • new message count = The number of new messages. • old message count = The number of old messages. • new urgent message count = The number of new messages marked urgent. • old urgent message count = The number of old messages marked urgent. Example: <code>^PHN-PRIVACY,0</code> The panel sent a command status to the master indicating there the call privacy is disabled.
^PHN-REDIAL	SIP call redial status - Indicates the panel is redialing the number. Syntax: <code>''^PHN-REDIAL,<number>''</code>

SIP Commands – Panel to Master	
	Variable: number = The phone number to dial. Example: <code>^PHN-REDIAL, 2125551000</code> The panel sent a command status to the master indicating the number 2125551000 is being redialed.
<code>^PHN-TRANSFERRED</code>	SIP call transferred status - Indicates a call has been transferred. Syntax: <code>''^PHN-TRANSFERRED,<connection id>''</code> Variable: connection id: The identifying number of the connection. Example: <code>^PHN-TRANSFERRED, 1</code> The panel sent a command status to the master indicating call 1 was transferred.

Master to Panel

The following table lists and describes SIP commands that are sent to the touch panel to manage calls.

SIP Commands - Master to Panel	
<code>^PHN-ANSWER</code>	SIP call answer command - Answers the call. Syntax: <code>''^PHN-ANSWER,<connection id>''</code> Variable: connection id = The identifying number of the connection Example: <code>SEND_COMMAND Panel, ''^PHN-ANSWER, 1''</code> Answer call 1.
<code>^PHN-AUTOANSWER</code>	SIP set auto-answer state command - Enables (1) or disables (0) the auto-answer feature on the phone. Syntax: <code>''^PHN-AUTOANSWER,<state>''</code> Variable: state = 0 (Disable) or 1 (Enable) Example: <code>SEND_COMMAND Panel, ''^PHN-AUTOANSWER, 1''</code> Enable the auto-answer feature.
<code>?PHN-AUTOANSWER</code>	Get SIP auto-answer state command - Queries the state of the auto-answer feature. The panel responds with the <code>^PHN-AUTOANSWER, <state></code> message. Syntax: <code>''?PHN-AUTOANSWER''</code> Example: <code>SEND_COMMAND Panel, ''?PHN-AUTOANSWER''</code> Get the auto-answer status.

SIP Commands - Master to Panel	
^PHN-CALL	SIP call command - Calls the provided number. Syntax: <code>''^PHN-CALL, <number>''</code> Variable: number = The provided phone number Example: <code>SEND_COMMAND Panel, ''^PHN-CALL, 2125551000''</code> Call the number 2125551000.
^PHN-DECLINE	Decline (send to voice mail if configured) the incoming call on <CallID> as indicated from the previous PHN-INCOMING message. CallID should be 0 or 1. Syntax: <code>''^PHN-DECLINE, <CallID>''</code> Variable: CallID = The identifying number of the connection. Example: <code>SEND_COMMAND Panel, ''^PHN-DECLINE, 0''</code> Decline the call with ID of 0.
^PHN-DTMF	SIP send DTMF tone command - Sends DTMF codes to an existing call. Syntax: <code>''^PHN-DTMF, <DTMF code>''</code> Variable: DTMF code = 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, POUND, or ASTERISK. Example: <code>SEND_COMMAND Panel, ''^PHN-DTMF, 5''</code> Send the DTMF tone for 5.
^PHN-HANGUP	SIP hangup call command - Hangs up the call. Syntax: <code>''^PHN-HANGUP, <connection id>''</code> Variable: connection id = The identifying number of the connection Example: <code>SEND_COMMAND Panel, ''^PHN-HANGUP, 1''</code> Hangup the call with ID of 1.
^PHN-HOLD	SIP put call on hold command - Places the call on hold. Syntax: <code>''^PHN-HOLD, <connection id>''</code> Variable: connection id = The identifying number of the connection Example: <code>SEND_COMMAND Panel, ''^PHN-HOLD, 1''</code> Put the call with ID of 1 on hold.

SIP Commands - Master to Panel	
^PHN-IM	SIP send instant message – Sends an instant message to another SIP device. Syntax: <code>''^PHN-IM,<target>,<msg>''</code> Variables: - target = The number or name of the SIP device the message should be sent. - msg = A short message no longer than 256 bytes. Example: <code>^PHN-IM,9002,A short message</code> Sends the message “A short message” to the device with the number 9002.
?PHN-LINESTATE	Get SIP linestate command - Queries the state of each of the connections used by the SIP device. The panel responds with the ^PHN-LINESTATE message. Syntax: <code>''?PHN-LINESTATE''</code> Example: <code>SEND_COMMAND Panel, ''?PHN-LINESTATE''</code> Get the current line states.
^PHN-PRIVACY	SIP set privacy state command - Enables or disables the privacy feature on the phone (do not disturb). Syntax: <code>''^PHN-PRIVACY,<state>''</code> Variable: state = 0 (Disable) or 1 (Enable) Example: <code>SEND_COMMAND Panel, ''^PHN-PRIVACY,1''</code> Enables the privacy feature.
?PHN-PRIVACY	Get SIP privacy state command - Queries the state of the privacy feature. The panel responds with the ^PHN-PRIVACY, <state> message. Syntax: <code>''?PHN-PRIVACY''</code> Example: <code>SEND_COMMAND Panel, ''?PHN-PRIVACY''</code> Get the current SIP privacy status.
^PHN-REDIAL	SIP call redial command - Redials the last number. Syntax: <code>''^PHN-REDIAL''</code> Example: <code>SEND_COMMAND Panel, ''^PHN-REDIAL''</code> Redial the last number.

SIP Commands - Master to Panel	
^PHN-TRANSFER	SIP call transfer message - Transfers the call to the provided number. Syntax: <code>''^PHN-TRANSFER,<connection id>,<number>''</code> Variables: - connection id: The identifying number of the connection - number: The number to which you want to transfer the call. Example: <code>SEND_COMMAND Panel, ''^PHN-TRANSFER,1,2125551000''</code> Transfer call with ID 1 to 2125551000.
^PHN-SETUP-DOMAIN	Set SIP domain name command - Set the domain name for the SIP server. Syntax: <code>''^PHN-SETUP-DOMAIN,<domain name>''</code> Variables: domain name: The domain name to use for the sip connection. Example <code>SEND_COMMAND Panel, ''^PHN-SETUP-DOMAIN,sip.domain''</code> Set the SIP domain to sip.domain
^PHN-SETUP-ENABLE	Enable SIP setup command - Registers a new user. Once the configuration has been updated, the ENABLE command should be run to re-register the new user. Syntax: <code>''^PHN-SETUP-ENABLE''</code>
^PHN-SETUP-PASSWORD	Setup SIP password command - Sets the user password so this extension can connect to the SIP server (SIP proxy server). Syntax: <code>''^PHN-SETUP-PASSWORD,<password>''</code> Variable: password: The password for the user name Example: <code>SEND_COMMAND Panel, ''^PHN-SETUP-PASSWORD,6003''</code> Setup the password for this extension to 6003.
^PHN-SETUP-PORT	Setup port for SIP Server connection command - Sets the port number for the proxy server. Syntax: <code>''^PHN-SETUP-PORT,<port>''</code> Variable: port: The port for the proxy server Example: <code>SEND_COMMAND Panel, ''^PHN-SETUP-PORT,5060''</code> Set this extension to connect to the SIP server (SIP proxy address) to port 5060.

SIP Commands - Master to Panel	
^PHN-SETUP-PROXYADDR	Setup SIP server address command - Sets the IP address for the SIP server (SIP proxy address). Syntax: <code>"^PHN-SETUP-PROXYADDR,<IP>"</code> Variable: IP: The IP address for the proxy server Example: <code>SEND_COMMAND Panel, "^PHN-SETUP-PROXYADDR,192.168.223.111"</code> Set the extension to try the SIP server (SIP proxy address) at the IP of 192.168.223.111.
^PHN-SETUP-USERNAME	Setup SIP username command - Sets the user name for authentication with the SIP server (SIP proxy address). Syntax: <code>"^PHN-SETUP-USERNAME,<username>"</code> Variable: username: The user name (usually the phone extension) Example: <code>SEND_COMMAND Panel, "^PHN-SETUP-USERNAME,6003"</code> Set the extension to authenticate to the SIP server with the username of 6003.

Commands similar to *TPControl*

The following resources are provided to document some special commands unique for **TPControl** and implemented in **TPanel**. The commands support features not available with the standard command set as defined by *AMX*.

All commands are case insensitive.

Commands similar to TPCControl	
TPCCMD-LocalHost	Set the NetLinx Master Connection IP/URL. Syntax: <code>TPCCMD-LocalHost,<IPAddress>[:PortNumber]</code> Variable: - IPAddress: The name or IP address of the AMX controller (NetLinx). - PortNumber: The optional network port number. Example: <code>TPCCMD-LocalHost,192.168.10.11:1319</code> Sets the network address of the controller to 192.168.10.11 and the network port number to 1319.
TPCCMD-LocalPort	Set the ICSP port number value (default port value is: 1319). Syntax: <code>TPCCMD-LocalPort,<PortNumber></code> Variable: PortNumber: The network port number. Example: <code>TPCCMD-LocalPort,1319</code> Sets the network port number to 1319.

Commands similar to TPControl	
TPCCMD-DeviceID	Set the Device ID number used upon connection to the NetLinx master. Syntax: TPCCMD-DeviceID, <value> Variable: value: A value in the range 10000 to 29999. Example: TPCCMD-DeviceID, 11001 Sets the channel of the panel to 11001.
TPCCMD-ApplyProfile	Provides the ability to recall the Settings stored. If the profile is different from the current active profile, TPanel will disconnect the active connection, and attempt to connect using the new profile settings. Syntax: TPCCMD-ApplyProfile Example: TPCCMD-ApplyProfile
TPCCMD-QueryDeviceInfo	Returns a STRING including related device identification information. Syntax: TPCCMD-QueryDeviceInfo Example: DeviceInfo-Tpanel, Androidi;HostName, MyLinux;uuid, <UUID>
TPCCMD-LockRotation	Enable or disable screen rotation. If screen rotation is not locked, the screen rotates only with the same orientation. This means: If the surface is set to portrait, for example, the screen can rotate only between normal portrait and reverse portrait (top down). Syntax: TPCCMD-LockRotation, <true false> Variable: true false: if this is true the screen is locked and will not rotate. Otherwise the screen will rotate. Example: TPCCMD-LockRotation, false Unlocks screen rotation.
TPCCMD-ButtonHit	When enabled, Button Hit produces a “Beep” sound when a valid button area is pressed within the touch panel design file. Syntax: TPCCMD-ButtonHit, <true false>; Variable: true false: If this is set to true, a beep is played, otherwise not. Example: TPCCMD-ButtonHit, true Enables the button beep.

Commands similar to TPControl	
TPCCMD-ReprocessTP4	Clears any caching, and reprocesses the installed TP4 file. This is the same process that runs whenever a file is transferred to the device. Syntax: TPCCMD-ReprocessTP4 Example: TPCCMD-ReprocessTP4
TPCACC	Used to return device orientation data, parsed to the controller in string format on Port 1. This command returns the screen orientation received from an orientation sensor in the device. This is independent from any set orientation in the TP4 surface file or the actual visible screen orientation of the surface. Syntax: TPCACC-<ENABLE DISABLE QUERY> Variables: • ENABLE: orientation data will be actively returned upon change of device orientation. • DISABLE: orientation data will stop being actively issued. • QUERY: returns the current device orientation. Response: TPCACC-<orientation> Where <orientation> can be: • DeviceOrientationPortrait • DeviceOrientationPortraitUpsideDown • DeviceOrientationLandscapeLeft • DeviceOrientationLandscapeRight Example: TPCACC-ENABLE Enables the transmission of device orientation data and sends the actual screen orientation to the controller.

TPanel specific commands

The following section shows commands only available for Tpanel.

GPS coordinates and maps

TPanel is able to get the GPS coordinates if it is running on a device which supports it. There exists commands to get the current coordinates as a custom event. TPanel is able to define a button showing a map. Such a button can be created over commands. Optional it is possible to select different sources for the map. It is possible to mark a point on a map and move the map to show the point in the center.

GPS coordinates and maps																			
*GPS	Retrieves the GPS coordinates from the device TPanel is running on. If the device supports GPS coordinates, this command returns the longitude and the latitude as a string in field <i>Text</i>. If GPS coordinates are not supported, the string “0.0,0.0” is returned. The format of the custom event transmitted to the controller is as follows: <table><tr><th>Custom Event Property</th><th>Value</th></tr><tr><td>Port</td><td>Port command was received on</td></tr><tr><td>ID</td><td>0</td></tr><tr><td>Type</td><td>3000</td></tr><tr><td>Flag</td><td>0</td></tr><tr><td>Value 1</td><td>0</td></tr><tr><td>Value 2</td><td>0</td></tr><tr><td>Value 3</td><td>0</td></tr><tr><td>Text</td><td>A string containing the coordinates as floating point numbers separated by a comma.</td></tr></table> Syntax: *GPS Example: *GPS Returns a custom event containing the GPS coordinates as a string in field Text: 48.199453,16.328604 Where 48.199453 is the longitude and 16.328604 is the latitude.	Custom Event Property	Value	Port	Port command was received on	ID	0	Type	3000	Flag	0	Value 1	0	Value 2	0	Value 3	0	Text	A string containing the coordinates as floating point numbers separated by a comma.
Custom Event Property	Value																		
Port	Port command was received on																		
ID	0																		
Type	3000																		
Flag	0																		
Value 1	0																		
Value 2	0																		
Value 3	0																		
Text	A string containing the coordinates as floating point numbers separated by a comma.																		
+MAP																			
+AMP																			

Appendix A

Text area input masking

Text Area Input Masking may be used to limit the allowed/correct characters that are entered into a text area. For example, in working with a zip code, a user could limit the entry to a max length of only 5 characters; with input masking, this limit could be changed to 5 mandatory numerical digits and 4 optional numerical digits. A possible use for this feature is to enter information into form fields. The purpose of this feature is to:

- Force the use of correct type of characters (i.e. numbers vs. characters)
- Limit the number of characters in a text area
- Suggest proper format with fixed characters
- Right to Left
- Required or Optional
- Change/Force a Case
- Create multiple logical fields
- Specify range of characters/number for each field

With this feature, it is not necessary to:

- Limit the user to a choice of selections
- Handle complex input tasks such as names, days of the week, or month by name
- Perform complex validation such as Subnet Mask validation

~~Input mask character types~~

~~These character types define what information is allowed to be entered in any specific instance. The following table lists what characters in an input mask will define what characters are allowed in any given position.~~